Israa University

Department of Engineering

Artificial Intelligence Engineering

A Data-Driven Approach to Emergency Dispatching Based on Response Time Prediction.

A Graduation Project Submitted in Partial Fulfillment of the Requirements for the Degree of Bachelor in Artificial Intelligence Engineering.

Prepared by:

Afnan Shomar 20220810

Raghad Abu Shammala 20230467

Razan Mortaja 20220762

Saja Mansour 20220685

Supervised by:

Dr.Ahmed Al-Salibi

Academic Year: 2025–2026

Abstract

This study presents a data-driven approach to improving emergency dispatching decisions through accurate prediction of response time in Emergency Medical Services (EMS). In high-pressure environments such as the Gaza Strip, where infrastructure damage, resource shortages, and unpredictable conditions affect operations, traditional dispatch methods based on proximity are insufficient. The research proposes an intelligent decision-support system that utilizes machine learning techniques, including XGBoost, CatBoost, and Random Forest, to predict emergency response time based on multiple dynamic factors such as traffic conditions, weather, incident severity, and resource availability. The system evaluates different response options and selects the optimal one based on predicted arrival time. A publicly available dataset (IERAD) is used due to the lack of reliable local data. The methodology includes data preprocessing, feature engineering, model training, and evaluation using metrics such as MAE, RMSE, and R^2. Additionally, the study considers extreme scenarios using advanced modeling techniques to better understand delays in critical cases. The results demonstrate that data-driven models can significantly enhance dispatch decision-making, improve response efficiency, and potentially save lives. This research contributes to the development of intelligent EMS systems capable of adapting to complex and resource-constrained environments.

Table of Contents

Chapter One

1.1 Introduction

Designing effective emergency response systems (EMS) for a wide range of incidents, including road accidents and large-scale natural disasters such as hurricanes, earthquakes, and wildfires, as well as human-induced emergencies such as wars, has become a critical challenge for communities worldwide. These systems play a vital role in protecting human life by ensuring rapid and coordinated responses to emergency situations. The efficiency of EMS operations directly impacts survival rates, making response time one of the most important performance indicators in emergency management systems. The EMS cycle begins with receiving an emergency call reporting an incident, followed by a dispatch decision in which the most appropriate response unit is selected and deployed at the optimal time. The dispatched unit then travels to the incident location, provides initial on-site medical care, and transports the injured individual to the most appropriate hospital for further treatment. During this process, route selection plays a critical role in minimizing travel time and ensuring timely arrival at the medical facility. After patient delivery, the ambulance is reassigned based on system demand and operational forecasts, either returning to the main station or relocating to another strategic position to improve readiness for future incidents. Generally, the ultimate goal of EMS is to save lives and minimize the impact of health-related incidents. Once a call is received, the ambulance assigned to the call must be selected quickly to ensure that medical assistance reaches the patient as soon as possible. The commonly used rule of selecting the nearest available ambulance is widely adopted in both practical EMS operations and research studies. However, this approach does not always guarantee the fastest response, as it fails to consider various dynamic factors that influence travel time and system performance. In real-world environments, response time is affected by multiple variables, including traffic congestion, weather conditions, road infrastructure, and resource availability. Therefore, relying solely on geographical distance is insufficient for making optimal dispatch decisions. As the nearest ambulance or drone does not necessarily guarantee the fastest arrival, there is a growing need for intelligent systems that can analyze

multiple factors simultaneously and support more accurate and efficient decision-making processes. With the advancement of data-driven technologies and machine learning techniques, it has become possible to develop predictive models that estimate emergency response time based on historical and contextual data. These models provide an opportunity to improve dispatch strategies by selecting the response option that minimizes delay and enhances overall system efficiency. As a result, integrating machine learning into emergency response systems represents a promising direction for improving operational performance and saving lives.

1.2 Research Motivation

Our focus on the emergency response situation in the Gaza Strip is motivated by the extreme operational challenges observed following the outbreak of war on October 7, when the healthcare system experienced a surge in demand that far exceeded its capacity. During this period, a large number of attacks occurred within short time intervals, resulting in injuries to approximately fifty to one hundred people within minutes. These incidents required the rapid deployment of between fifty and eighty ambulances per event, placing unprecedented pressure on emergency medical services. This extraordinary demand exposed significant weaknesses in the existing emergency response system. Serious organizational challenges emerged, particularly in coordinating resources, prioritizing emergency incidents, and ensuring timely response under highly constrained conditions. The overwhelming number of casualties also exceeded the capacity of healthcare facilities, including radiology units, laboratories, and emergency departments, which significantly hindered the effective handling of distress calls and patient care.

Furthermore, the destruction of infrastructure and the damage or loss of ambulances during the conflict critically reduced transportation capacity. This led to severe limitations in the availability of response units, further increasing delays in reaching emergency scenes. In such environments, traditional dispatching methods, which rely primarily on selecting the nearest available ambulance, become inefficient and inadequate for handling complex and rapidly changing conditions. In addition to operational challenges, the unpredictable nature of the environment, including blocked

roads, unstable communication networks, and fluctuating resource availability, adds further complexity to decision-making processes. These conditions highlight the urgent need for intelligent and adaptive systems that can support dispatch decisions based on real-time and historical data rather than simple static rules. What inspires this research is not only the technical challenges but also the humanitarian perspective. According to the principle of humanity proposed by Immanuel Kant (Kant, 1785), every human being should be treated as an end in themselves rather than merely as a means. In the context of emergency response, this means that every call represents a life that must be treated with urgency, care, and dignity. Therefore, improving response efficiency is not only a technical problem but also a moral responsibility. These challenges collectively motivate the development of a data-driven approach that utilizes machine learning techniques to predict emergency response time and support optimal dispatch decisions. By improving the accuracy of response time estimation and considering multiple influencing factors, this research aims to enhance the efficiency, fairness, and effectiveness of emergency medical services, particularly in high-risk and resource-constrained environments such as the Gaza Strip.

1.3 Problem statement

In war-affected regions such as Gaza, Emergency Medical Services (EMS) operate under extreme conditions. Based on the available data, predicting response time is critical when infrastructure is damaged, roads are blocked, and communication networks are unstable. In such environments, ambulance dispatching decisions directly affect how quickly medical assistance can arrive. Geographical factors, limited ambulance availability, and hospital capacity constraints all play a significant role in response time and patient outcomes. As shown in the figure, the EMC cycle consists of multiple stages.

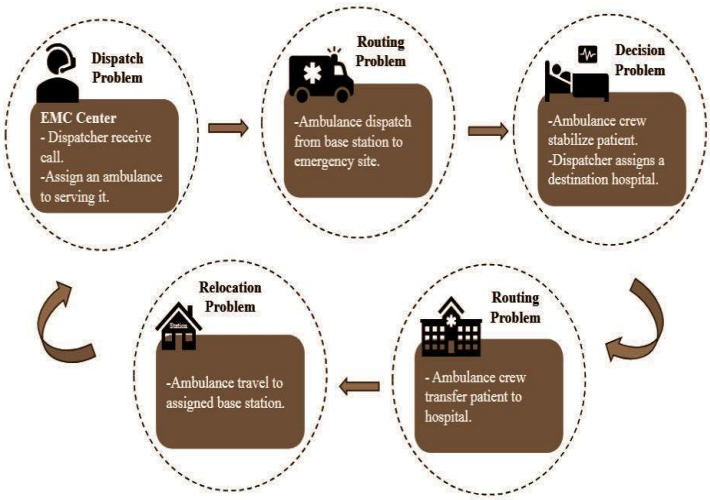


Fig. 1. EMC Lifecycle.

In this study, we focus on the first stage, dispatching. When a call arrives at the EMS center, its timestamp, region type, incident severity, and incident type are recorded. Using this dispatch-time information, including traffic congestion, weather conditions, road type, and resource availability, the dispatcher selects an appropriate response option. If no suitable resources are available, the call is delayed until a response unit becomes available. Dispatch decisions are influenced by factors such as ambulance and drone availability, vehicle speeds, battery life, fuel level, and hospital capacity. Once a response option is selected, the corresponding team is notified and dispatched. At the incident scene, medical assistance is provided as required. If hospital transport is necessary, the patient is transferred to a hospital subject to available capacity. After completing the mission, the response resource becomes available for subsequent dispatch decisions. Despite the availability of rich datasets, there remains a lack of AI-based decision support systems capable of accurately predicting emergency response time and analyzing how factors such as incident type, regional context, distance to the incident, ambulance availability, and hospital capacity influence that response. Addressing this gap is critically important, as even small improvements in response efficiency can translate into lives saved. However, due to the current situation in the Gaza Strip, access to reliable real-time emergency data is not feasible. Therefore, this study utilizes a publicly available dataset obtained from Kaggle as an alternative data source. It is acknowledged that some features in the dataset may not fully reflect real-world conditions in Gaza, as information available from humanitarian and NGO sources

is limited. Nevertheless, the dataset provides a sufficiently large and structured foundation for developing and evaluating predictive models. Consequently, the analysis is conducted using the available data, with the intention that future work may incorporate scenario-based simulations, such as Monte Carlo methods, to better approximate real-world conditions in Gaza. This research uses the Integrated Emergency Response Analytics Dataset (IERAD), which contains 368,065 emergency records collected between 2018 and 2024 in Sydney, Australia. The dataset integrates data from ambulance dispatch units, drone operators, and regional hospitals, providing a comprehensive view of emergency response operations. Accordingly, the main problem we will discuss in this research is the emergency dispatching problem within emergency systems.

1.4 Research Objectives

Our main objective is to develop a data-driven approach for emergency dispatching that selects the response option ambulance, drone, or hybrid with the minimum predicted response time.

Sub objectives:

- To build and train a machine learning model to predict emergency response time using dispatch time information.
- To evaluate and compare response options based on predicted response times.
- To incorporate emergency priority levels into the dispatching decision process.
- To analyze the effect of operational and environmental factors on response time.

1.5 Methodology

After the data cleaning process, which will include handling missing values, standardizing data formats, and treating outliers, machine learning models will be applied to predict emergency response time. For XGBoost, categorical variables will be transformed using one-hot encoding, while CatBoost will be employed directly on the original categorical features due to its native handling of categorical data and its expected superior performance on mixed-type data. These models will be trained to estimate response time based on incident characteristics and dispatch options. To avoid

optimizing only the mean response time and to reduce the impact of extreme delays, quantile regression will be incorporated to model the upper tail of the response time distribution. Specifically, the 90th conditional quantile of response time will be predicted for each dispatch option (ambulance, drone, and hybrid), and the optimal dispatch decision will be determined by :

$$d^* = \arg \min_{d \in \{\text{Drone}, \text{Ambulance}, \text{Hybrid}\}} \hat{t}_d^{(90)}$$

This approach will help to understand how different factors affect different parts of the response time distribution and will provide better insight into extreme response scenarios.

1.6 Scope

This study focuses on the use of machine learning methods to predict emergency response time and support dispatch decisions in pre-hospital Emergency Medical Services (EMS) operations. Within this scope, dispatch decisions are analyzed using dispatch-time features such as traffic conditions, weather, distance to the incident, and temporal information. The analysis is limited to historical EMS data and offline model evaluation, and does not include real-time system deployment. Also data augmentation is not applied, as the available dataset is sufficiently large to support reliable model training and evaluation, and the introduction of synthetic data may distort real operational patterns.

Chapter Two -Literature Review

2.1 Introduction

Emergency Medical Services (EMS) constitute a critical component of any effective healthcare system, as their performance is directly associated with patient survival and community resilience. Among the various performance indicators used to evaluate EMS operations, response time remains one of the most sensitive and consequential. Even minor delays can significantly affect clinical outcomes, particularly in time-critical conditions. As emergency environments become increasingly complex characterized by fluctuating demand, resource constraints, and operational uncertainty the need for more intelligent and data-driven dispatch mechanisms becomes evident. Traditional dispatch systems typically rely on predefined rules, such as assigning the nearest available ambulance or following fixed priority classifications. While such approaches may provide acceptable performance in stable settings, they often fail to account for dynamic operational factors, including variations in workload, resource availability, and contextual constraints. Consequently, decision-making processes may not fully reflect the real-time complexity of emergency response environments (Bélanger et al., 2015). In response to these challenges, recent research has increasingly explored the application of artificial intelligence and machine learning techniques to enhance emergency response performance (Hill et al., 2025; Shahidian et al., 2025). Data-driven models have been developed to analyze historical call records, predict response times, forecast demand patterns, and support resource allocation decisions (Lucas, 2026). While these studies demonstrate improvements in predictive capability, their application in practical and operational decision-support systems remains limited (Alrawashdeh et al., 2024). This chapter establishes the theoretical and empirical foundation for the present study. It first outlines the operational structure of EMS systems and describes the lifecycle of an emergency mission, highlighting key decision points and sources of complexity. It then reviews relevant literature on response-time modelling, demand forecasting, and intelligent dispatch optimization. Through this structured discussion, the chapter positions the current project within the existing body

of research and provides the conceptual basis for the proposed data-driven decision support system developed in the subsequent chapters.

2.2 Background

2.2.1 Emergency Medical Services and System Efficiency

The efficiency of Emergency Medical Services (EMS) systems plays a crucial role in strengthening community resilience and improving disaster management capabilities. These systems operate in a wide range of situations, from everyday incidents such as road accidents and medical emergencies to large-scale events including natural disasters, armed conflicts, and humanitarian crises. In all such scenarios, the effectiveness of emergency response particularly in terms of speed and coordination—has a direct impact on reducing mortality rates, preventing long-term health complications, and minimizing both economic and social losses. One of the most important indicators used to evaluate EMS performance is response time, which refers to the duration between receiving an emergency call and the arrival of medical assistance at the scene. Research consistently shows that shorter response times are closely linked to higher survival rates, especially in critical situations such as trauma, cardiac arrest, and severe bleeding (Hill et al., 2025). This is further emphasized by the concept of the "golden hour," which highlights the importance of providing timely medical care within a critical period to significantly increase the chances of survival (Lerner & Moscati, 2001). To better understand EMS operations, the system can be analyzed through a structured lifecycle model that represents the sequential stages of emergency response, from call reception to resource deployment and patient care (Bélanger et al., 2015).

2.2.2 The Emergency Medical Center Lifecycle (EMC Lifecycle)

The Emergency Medical Center Lifecycle (EMC Lifecycle) describes the sequence of operational phases that occur from the moment an emergency call is received until the system returns to a state of readiness for future incidents. This lifecycle can be divided into several interconnected stages.

1.The Dispatch Problem

The lifecycle begins with the dispatch phase, which is initiated immediately upon receiving a distress call. During this stage, critical information is collected, including the time of the call, geographical location, nature of the incident, and its severity. Based on this information, the system must determine the most appropriate response unit to allocate. The dispatch problem is inherently complex and dynamic. It is influenced by multiple factors such as traffic congestion, weather conditions, vehicle availability, fuel levels, and hospital capacity. The decision-making process may be conducted by human operators, supported by decision-support systems, or increasingly by AI-assisted tools. The goal at this stage is to select a response strategy that minimizes expected response time while ensuring appropriate medical capability.

2. Tactical Execution and the Routing Problem (Deployment Phase)

Once a response unit has been assigned, the operation enters the deployment phase. The selected unit travels from its current location to the incident site. At this stage, route selection becomes a critical factor in minimizing travel time. The routing problem involves identifying the most efficient path under real-world constraints, including road networks, congestion levels, and potential obstacles. Effective routing strategies aim to reduce travel delays and ensure rapid arrival at the scene.

3. Field Care and On-Site Decision Making

Upon arrival at the incident location, medical personnel provide immediate care and stabilize the injured individual. This phase may involve triage, initial assessment, and life-saving interventions. A subsequent decision must then be made regarding whether the patient requires transfer to a specialized medical facility. This decision depends on clinical condition, available hospital capacity, and the level of care required.

4. Hospital Transfer and Secondary Routing

If patient transfer is necessary, the operation enters a second routing phase. The response unit transports the patient from the incident site to the designated hospital. As in the initial deployment phase, minimizing travel time remains critical. Arrival within the clinically optimal treatment window often referred to as the golden hour can significantly influence survival and recovery outcomes. Therefore, routing efficiency continues to be a central operational concern.

5. System Readiness and Relocation

After completing the mission and delivering the patient, the response unit becomes available for future calls. This phase involves restoring system readiness. The unit may return to its base location or be strategically repositioned in areas with higher predicted demand. Relocation decisions influence overall system coverage, preparedness, and the ability to respond effectively to subsequent emergencies.

2.2.3 Operational Complexity in High-Stress Environments

While the EMC Lifecycle provides a structured representation of EMS operations, real-world environments often introduce additional layers of complexity. In high-stress or resource-constrained settings such as natural disasters, large-scale accidents, or conflict-affected regions road infrastructure damage, communication disruptions, and sudden surges in casualty numbers can severely strain system capacity.

Under such conditions, predicting response performance becomes more challenging, and efficient allocation of limited resources becomes critically important. These environments highlight the necessity of systematic, data-informed approaches to support dispatch and routing decisions under uncertainty.

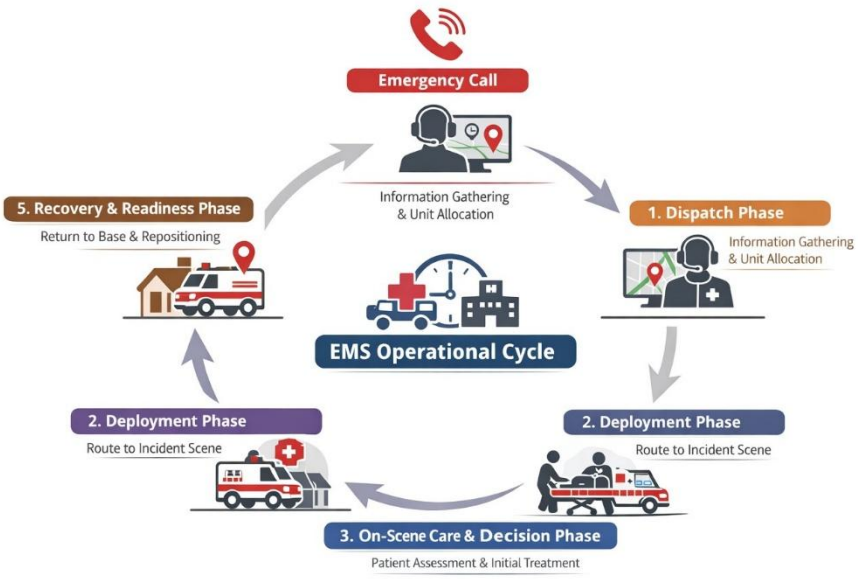


Figure 2.1: EMS life-cycle

2.3 Related Works

This section reviews recent studies related to emergency response systems, with a focus on response time prediction, demand forecasting, and intelligent dispatching strategies. The reviewed works explore the use of machine learning and optimization techniques to improve decision-making in Emergency Medical Services (EMS). Particular attention is given to the algorithms and models proposed in each study, highlighting their methodologies, contributions, and limitations.

2.3.1 Gradient Boosting Model for EMS Response Time Prediction

Hill et al. (2025) addressed the problem of variability in Emergency Medical Services (EMS) response times by developing a machine learning-based framework using a large dataset of over one million emergency missions collected in Stockholm between 2017 and 2022. The primary objective of the study was to identify the operational, environmental, and geographical factors that significantly influence response time in order to support more efficient resource allocation and improve system performance.

The authors proposed a Gradient Boosting model as the main predictive approach. This model integrates multiple categories of features, including call-level information such as priority level, incident type, and travel distance; system-level indicators such as workload and mission frequency; and environmental variables such as temperature and precipitation. This combination of features enables the model to capture complex and non-linear relationships affecting response time. From a methodological perspective, the study compared several predictive models, including linear regression, random forests, neural networks, and gradient boosting. The dataset was divided into training, validation, and testing subsets to ensure reliable evaluation. Among all models, gradient boosting achieved the best performance in terms of prediction accuracy, demonstrating its effectiveness in handling large-scale and heterogeneous EMS data. To enhance interpretability, the authors used feature importance analysis and partial dependence plots to examine how different factors influence response time. The results showed that dispatch priority is the most significant determinant, followed by workload conditions and travel distance. Additionally, external factors such as weather conditions were found to have a measurable impact on response delays. The study also highlighted an

important data characteristic: the response time distribution is highly skewed, particularly for lower-priority calls. Instead of removing extreme values, the authors retained these observations to better understand prolonged delays, which are operationally critical.

Despite its strengths, the study has several limitations. It is based on data from a single metropolitan area, which may limit generalizability to other regions. Furthermore, real-time traffic information was not included, even though it is a key factor affecting travel time. Some variables were aggregated at the hourly level, potentially masking short-term fluctuations in system demand. In relation to the present study, Hill et al. (2025) focus on predicting average response time using gradient boosting techniques. In contrast, this research extends the analysis by examining the distribution of response times, with particular emphasis on extreme delays and their implications for emergency dispatch decision-making.

2.3.2 Hybrid Deep Learning and Reinforcement Learning for Intelligent Emergency Response

Lucas (2026) proposed an integrated framework for an Intelligent Emergency Response System (IERS) that combines Machine Learning (ML) with Internet of Things (IoT) technologies to enable proactive and adaptive emergency management. The study addresses the limitations of traditional emergency systems, which rely heavily on manual reporting, static data sources, and centralized coordination, often resulting in delayed response times and incomplete situational awareness. The proposed system is based on a distributed architecture designed to support real-time data collection, processing, and decision-making across multiple operational layers. A key innovation in this framework is the use of edge and fog computing to reduce latency by performing data processing closer to the source, thereby overcoming delays associated with centralized cloud-based systems. From an algorithmic perspective, the framework employs a hybrid machine learning approach. Convolutional Neural Networks (CNNs) are used for image-based detection and classification of incidents using surveillance cameras and drone data. Long Short-Term Memory (LSTM) networks are applied to model temporal dependencies in time-series data, enabling prediction of dynamic

events such as fire spread or flood progression. In addition, reinforcement learning is utilized to optimize routing decisions and resource allocation by adapting to changing traffic conditions and operational environments. The system also incorporates advanced techniques such as anomaly detection, multi-sensor data fusion, and federated learning to enhance data reliability and privacy. Furthermore, explainable artificial intelligence (XAI) is discussed as a mechanism to improve transparency and trust in automated decision-making processes. An important feature of the framework is its closed-loop operational cycle, which continuously integrates sensing, data transmission, analysis, decision-making, and feedback to improve system performance over time. The results of the study, demonstrated through several application scenarios such as fire detection, traffic accident management, and disaster response, indicate that the proposed approach can significantly improve detection speed, routing efficiency, and coordination among emergency services.

Despite its strengths, the framework has several limitations. It assumes the availability of advanced IoT infrastructure and reliable communication networks, which may not be feasible in resource-constrained or disaster-affected environments. Additionally, many of the deep learning models function as black-box systems, and although explainability is discussed, it is not extensively validated. The study is also largely conceptual, with limited quantitative evaluation or comparison with existing emergency response systems.

In relation to the present study, Lucas (2026) focuses on system architecture and real-time adaptive routing using hybrid learning techniques. In contrast, this research emphasizes response-time prediction and its integration into dispatch-level decision-making, providing a data-driven approach to improving emergency response performance. **2.3.3 Machine Learning Classification Models for EMS Demand Forecasting**

Shahidian et al. (2025) addressed the challenge of forecasting short-term ambulance demand in Emergency Medical Services (EMS) systems, which is influenced by increasing call volumes, limited resources, and the need to reduce response times. The study focuses on improving resource allocation, vehicle positioning, and staff

scheduling by providing more operationally meaningful demand predictions. The authors proposed a machine learning framework that reformulates the demand forecasting problem from a traditional regression task into a classification problem. Instead of predicting the exact number of emergency calls, demand is categorized into discrete levels, such as low and high demand, allowing predictions to align more closely with real-world decision-making processes. From an algorithmic perspective, the study evaluates multiple machine learning models, including Gaussian Mixture Models, Naïve Bayes, K-Nearest

Neighbors, Support Vector Machines, Random Forest, Gradient Boosting, AdaBoost, and Bagging. All models were tested under standardized conditions using cross-validation and consistent evaluation metrics such as accuracy, precision, and recall. The study also compares different feature selection approaches, including filter and wrapper methods, as well as hyperparameter tuning techniques such as Random Search and Genetic Algorithms. The results demonstrate that transforming demand forecasting into a classification problem improves operational usability by producing actionable demand categories. The study also highlights the importance of balancing predictive accuracy with computational efficiency when selecting machine learning models. Additionally, the framework enables the prediction of call priority levels and vehicle types, supporting integration into optimization systems for scheduling and resource allocation. Despite these contributions, several limitations are identified. Discretizing demand into categories may obscure variability within each group, potentially reducing prediction granularity. Furthermore, the study does not evaluate the direct impact of the proposed models on response time performance or patient outcomes. Although priority levels are predicted, they are not explicitly incorporated into dispatch decision-making processes.

In relation to the present study, Shahidian et al. (2025) focus on aligning demand forecasting with operational categories through classification-based modelling. In contrast, this research emphasizes response-time performance and integrates emergency priority directly into the decision-making framework for resource allocation.

2.3.4 Logistic Regression Model for Spatial Temporal Ambulance Demand Prediction

Kerakos et al. (2020) investigated the problem of predicting ambulance demand across different regions and time periods in order to support dynamic deployment and improve system readiness in Emergency Medical Services (EMS). The study focuses on capturing spatial and temporal variations in demand to enhance decision-making related to ambulance positioning. The authors proposed a data-driven approach that combines clustering techniques with a binary Logistic Regression model. Instead of relying on predefined administrative boundaries, clustering was used to group geographical areas with similar demand patterns, improving the balance between spatial resolution and data reliability. This approach allows the model to better represent real-world demand distribution. From a methodological perspective, the Logistic Regression model was designed to predict whether at least one ambulance dispatch would occur within a specific cluster during a given time interval. The model was trained and evaluated using separate training and test datasets to ensure proper validation. Its performance was compared with a historical baseline approach commonly used in EMS systems.

The results demonstrated that the Logistic Regression model outperformed the baseline method in terms of predictive accuracy. The study also revealed that ambulance demand varies significantly across both geographical areas and time periods, with distinct patterns observed between urban and suburban regions. Incorporating geographical features was found to improve prediction performance, and the authors highlighted that relying solely on average response time may overlook fairness in coverage across different locations.

Despite these contributions, several limitations were identified. The study is based on data from a single metropolitan region, which may limit its generalizability to other settings. Additionally, the model does not incorporate real-time traffic conditions or other dynamic external factors that could influence demand. As a linear model, Logistic Regression may also struggle to capture complex non-linear relationships. Furthermore, the study does not evaluate the real-world impact of the model on response times or patient outcomes. In relation to the present study, Kerakos et al. (2020) focus on

predicting the occurrence of ambulance demand using spatial–temporal modelling. In contrast, this research extends beyond demand prediction by analysing response-time variability and integrating predictive insights into performance optimization and dispatch decision-making.

2.3.5 Machine Learning and Ensemble Methods in Emergency Medical Services: A Scoping Review

Alrawashdeh et al. (2024) conducted a comprehensive scoping review to examine the application of machine learning techniques across Emergency Medical Services (EMS), covering both clinical and operational domains. The study addresses the growing use of machine learning in EMS while highlighting the lack of systematic evaluation and real-world implementation in prehospital settings. The review analyzes 164 studies published between 2005 and 2024 and categorizes them into clinical and operational applications. From an algorithmic perspective, the most frequently used models include Neural Networks, Random Forest, Support Vector Machines, Logistic Regression, and various ensemble methods. The study shows that advanced models, particularly neural networks and ensemble techniques such as Random Forest and Gradient Boosting, generally outperform traditional approaches in predictive tasks. The findings indicate that machine learning can significantly improve multiple EMS functions, including triage and diagnosis, prediction of clinical outcomes, ambulance allocation, deployment strategies, and route optimization. Performance metrics reported across studies demonstrate strong predictive capabilities, with high values of accuracy, sensitivity, specificity, and area under the curve. However, the results also reveal considerable variability depending on the dataset, application domain, and model complexity. The study further highlights that, despite strong theoretical performance, real-world implementation of machine learning models in EMS systems remains limited. Many studies focus primarily on predictive accuracy without addressing integration into live operational systems or clinical workflows. In addition, inconsistencies in model design, input features, and evaluation metrics make it difficult to compare results across studies. Several limitations are identified in the existing literature. These include a lack of standardization, limited generalizability due to geographical bias toward resource-rich

environments, and insufficient validation in real-world settings. The study also notes challenges related to model interpretability, particularly for deep learning approaches, as well as risks of bias and overfitting in highly accurate models. Furthermore, regulatory and governance considerations are often overlooked in current research. In relation to the present study, Alrawashdeh et al. (2024) emphasize the effectiveness of machine learning models in predicting EMS outcomes, with a primary focus on average predictive performance. In contrast, this research focuses on analysing extreme response-time delays and integrating predictive models into dispatch-level decision-making processes to improve operational efficiency.

2.4 Summary

Table 2.1 below provides a comparative summary of the most relevant studies reviewed in this section, highlighting the main algorithms, objectives, key contributions, and limitations of each work. The comparison shows that recent research has increasingly focused on applying machine learning techniques, particularly ensemble methods and deep learning models, to improve prediction accuracy and operational efficiency in EMS systems.

Table 2.1: Summary and Comparison of Related Works

Study	Main Algorithm/Model	Objective	Key Contribution	Limitations
Hill et al. (2025)	*Gradient Boosting*	Predict EMS response time	Identifies key factors (priority, workload, distance)	No real-time traffic data; limited to one city
Lucas (2026)	CNN, LSTM, *Reinforcement Learning*	Intelligent emergency detection & routing	Hybrid ML-IoT system with adaptive routing	Requires advanced infrastructure; mostly conceptual
Shahidian et al. 2025)	RF, SVM, GB, KNN *(Classification models)*	Forecast ambulance demand	Classification for better decision-making	Loss of detail due to categorization; no response-time evaluation
Gerakos et al. (2020)	*Logistic Regression + Clustering*	Predict spatial–temporal demand	Clustering for spatial modeling	Linear model; no real-time data; limited generalizability
Alrawashdeh et al. 2024)	Neural Networks, RF, SVM, *Ensemble*	Review ML applications in EMS	Comprehensive EMS ML analysis	*Limited real-world implementation;* lack of standardization

Chapter Three -Methodology

3.1 Introduction

This chapter presents the research methodology adopted to achieve the objectives of this study, which aims to develop a data-driven framework for guiding emergency response operations based on response time prediction. Given the complexity of Emergency Medical Services (EMS) and the influence of multiple dynamic factors on response performance, a quantitative, data-driven approach is employed to systematically model and analyze emergency response behavior. The chapter provides a comprehensive description of the research design, including data preprocessing, and feature engineering. Particular attention is given to addressing common data-related challenges, such as handling missing values, treating outliers, and representing categorical variables. These steps are essential to ensure the reliability, robustness, and generalizability of the predictive models. Furthermore, this study utilizes advanced machine learning techniques, particularly gradient boosting models such as CatBoost and XGBoost, to estimate emergency response times under varying operational conditions. These models are selected due to their strong capability in capturing complex, non-linear relationships and effectively handling mixed-type data.

3.2 Key Features of the Proposed System

The developed system serves as an intelligent technological framework that transforms raw emergency data into real-time operational decisions to support rescue teams. The application's features are designed in alignment with the Emergency Management Lifecycle (EMC) phases, with a particular emphasis on providing innovative solutions to the ambulance dispatching problem.

These features include:

1. Intelligent Automation for Receiving and Classifying Reports

- Dynamic Information Documentation: The system does not rely solely on traditional data; instead, it processes and transforms initial parameters into software features that are used to generate machine learning models. The

following table presents the core components handled within the system, along with their corresponding types and sources in the IERAD database: Key Input Features for the Emergency Response Prediction Model.

Feature	Description	Type	Justification
Distance_to_Incident	Distance between the dispatch point and th incident location.	Numerical	A primary determinant of travel duration and overall response time.
Traffic_Congestion	Level of road traffic at the time of dispatch.	Categorical/Ordina l	Directly influences ambulance delay and route efficiency.
Weather_Condition	Weather conditions during the dispatch per	Categorical	Affects the operational performance of both ambulance and drone services.
Ambulance_Availability	Availability status of an ambulance for immediate deployment.	Binary	Determines the feasibility of selecting ground transportation.
Drone_Availability	Availability status of a drone for dispatch.	Binary	Determines the feasibility of selecting aerial or hybrid transportation.
Battery_Life	Remaining battery capacity of the drone.	Numerical	Influences drone feasibility, endurance, and service reliability.
Ambulance_Speed	Estimated operational speed of the ambula	Numerical	Directly contributes to the prediction of ambulance response time.
Drone_Speed	Estimated operational speed of the drone.	Numerical	Directly contributes to the prediction of drone response time.

- Feature Extraction: The proposed system automatically extracts key operational and temporal features from the available data, such as weather conditions and traffic congestion, in order to provide the predictive models with reliable and contextually relevant inputs for response time estimation.

2. Advanced Predictive Response Time Modeling

- Multi-Algorithm Predictive Framework:

  The proposed system employs a multi-algorithm machine learning framework, incorporating models such as XGBoost, CatBoost, and Random Forest, to predict the response time of available rescue and transportation units. This approach enables the system to estimate expected response times under varying operational conditions with improved accuracy.

- Modeling of Nonlinear Relationships:

  These machine learning models are well suited for capturing complex and nonlinear relationships among operational variables, such as the interaction between road type and traffic congestion. As a result, they provide more accurate and robust response time predictions compared with conventional rule-based or linear prediction methods.

3. Optimal Dispatch Decision Support (d*)

- Dispatch Optimization through Comparative Evaluation:

  This component represents the core decision-support function of the proposed system. A mathematical optimization process is applied to evaluate the available response alternatives and identify the most appropriate dispatch option, denoted by d *, with the objective of minimizing predicted response time while maintaining operational feasibility.

- Hybrid Dispatch Strategy:

  The system assesses three alternative dispatch modes in order to determine the most effective response strategy for each emergency case:

  Ambulance Dispatch: Applied in situations requiring full medical stabilization, patient handling, and transportation to a healthcare facility.

- Drone Dispatch: Used to provide rapid initial response, preliminary assessment, or urgent medical supply delivery, particularly in situations where ground movement may be delayed.
- Hybrid Dispatch: A coordinated response strategy that combines drone speed with ambulance transport capability to improve intervention efficiency and maximize the likelihood of timely care during critical emergencies.

4. Intelligent Routing and Adaptation to Harsh Environments

- Obstacle-Aware Route Planning:

  The proposed system incorporates intelligent routing capabilities that support the selection of efficient response paths while accounting for physical obstacles that may delay emergency movement.

- Adaptation to the Gaza Strip Context:

  The routing mechanism is specifically designed to operate under the exceptional environmental and infrastructural conditions of the Gaza Strip. In particular, it considers damaged roads, blocked pathways, and isolated areas that may be difficult to access using ground vehicles. In such cases, the system can recommend drone-supported dispatch as an alternative response mechanism to improve accessibility and reduce delay.

5. Resource Management and Logistical Readiness

- Monitoring of Unit Operational Readiness:

  To enhance logistical efficiency, the system monitors the operational status of available response units, including ambulance drone battery life. This functionality supports better allocation and relocation decisions while helping maintain readiness for subsequent emergency requests.

6. Visual Analytics and Performance Evaluation

- Decision Support Dashboard:

  The system provides a visual decision-support interface that presents prediction outputs and dispatch recommendations in a clear and interpretable form. This contributes to improving situational awareness, particularly under high-pressure emergency conditions.

- Feedback and Performance Evaluation Loop:

  The system documents the key stages of the service cycle, allowing the performance of the predictive model to be evaluated and compared with historical data for ongoing refinement

  Collectively, these components enable the proposed system to function not only as a response time prediction tool, but also as an intelligent dispatch decision-support framework capable of optimizing emergency response operations under complex and resource-constrained conditions.

3.3 System Workflow

This study will develop a machine learning-based system for predicting emergency response time through a structured and sequential methodology. The goal of this system will be to transform raw emergency data into meaningful insights that can support decision-making in critical situations. The process will begin with the data acquisition stage, where the dataset will be obtained from a reliable external source, namely Kaggle.

The collected data will then be loaded into the system for further processing and analysis. In the next stage, data preprocessing will be performed. This step will involve cleaning the dataset, handling missing values, and transforming the data into a suitable format for machine learning algorithms. After preprocessing, the dataset will be split

into training and testing sets. The training set will be used to train the models, while the testing set will be used to evaluate their performance on unseen data. Subsequently, multiple machine learning models will be trained, including Random Forest, XGBoost, and CatBoost. Each model will learn the relationships between input features and response time . Following model training, model evaluation will be conducted using appropriate evaluation metrics to compare the performance of the models . Finally, the best-performing model will be selected and used for predicting response time for new emergency cases, thereby supporting decision-making in emergency response systems. After predicting the response time, the output of the model will be used to support dispatch decision-making. Specifically, the predicted response time can help prioritize emergency cases, estimate delays, and guide the allocation of available resources. For example, if the model predicts a long response time for a particular incident, decision-makers can take proactive actions such as reallocating resources, dispatching additional units, or prioritizing critical cases . In addition, the system can assist in identifying high-risk situations where delays are likely to occur, enabling better planning and faster response under constrained conditions, particularly in resource-limited environments This workflow ensures a logical and well-structured process, where each stage depends on the output of the previous one.

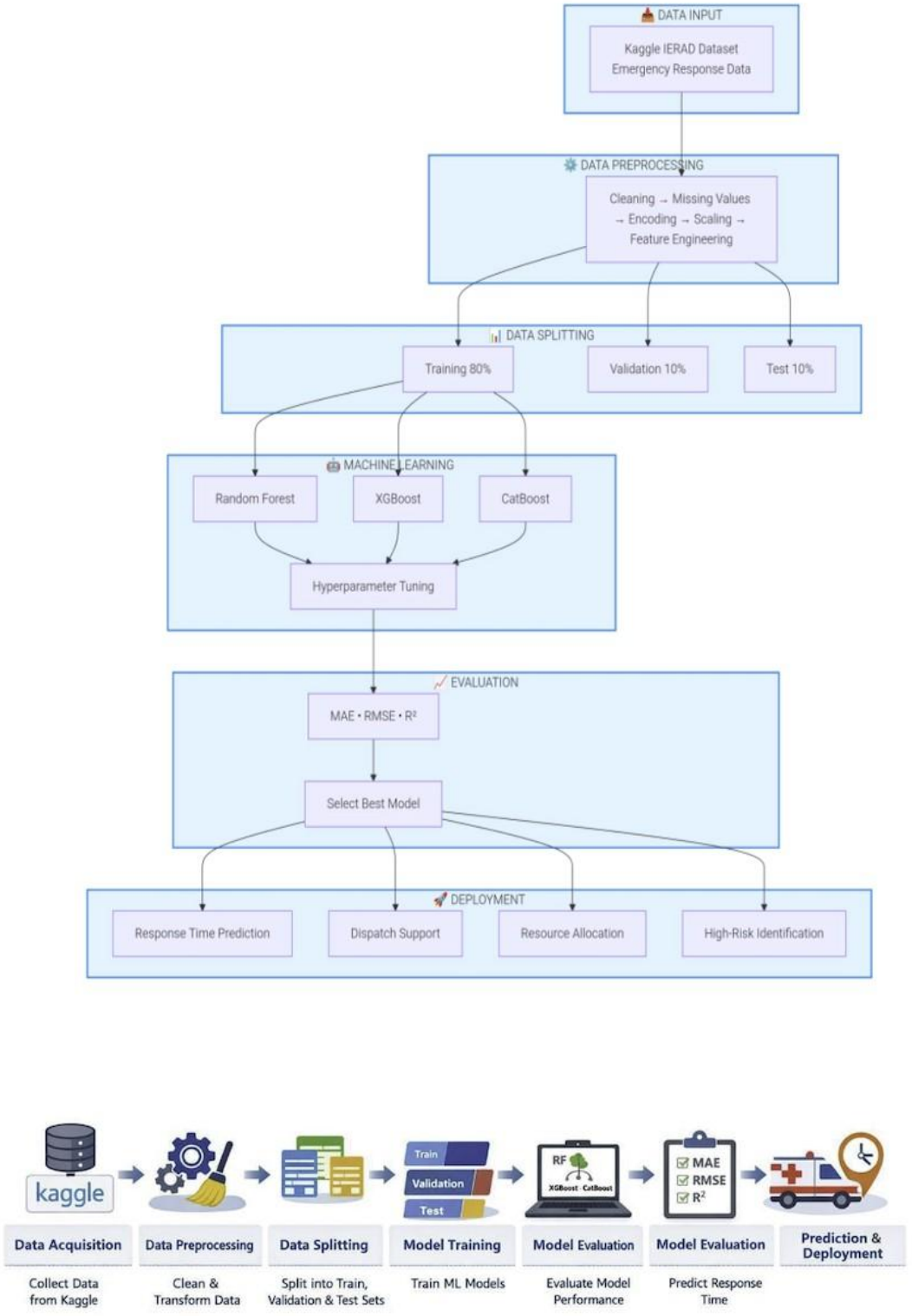


Figure 1: System Workflow for Emergency Response Time Prediction

3.4 Dataset Description and Justification

The dataset that will be used in this study is the Integrated Emergency Response Analysis Dataset (IERAD) obtained from Kaggle. This dataset contains structured records of emergency incidents and response operations, including key variables such as incident type, geographical location, severity level, resource allocation, dispatch time, arrival time, and total response time. The justification for selecting this dataset is a critical component of this research. Ideally, a locally sourced dataset from Gaza would have been more representative of the operational environment. However, the absence of reliable and structured emergency datasets from Gaza is mainly due to infrastructural limitations, restricted data accessibility, and the current Israeli war on Gaza. Therefore, an alternative approach will be adopted by utilizing a real-world dataset from Australia. Despite geographical differences, emergency response systems share fundamental operational characteristics, such as prioritization mechanisms, resource allocation strategies, and time-critical decision-making processes. Accordingly, this dataset will be treated as a proxy dataset, allowing the model to learn the relationships between incident attributes and response time. These learned patterns can later be conceptually applied to similar environments.

3.4.1 Data Understanding

Before applying machine learning techniques, a data understanding phase will be conducted to explore the structure and characteristics of the dataset.

The dataset will consist of both:

- Numerical variables, such as response time and resource availability
- Categorical variables, such as incident type and location

An Exploratory Data Analysis (EDA) process will be performed to:

1. Identify missing values
2. Detect outliers
3. Analyze data distributions
4. Explore relationships between variables

The target variable in this study will be Response Time, which will be predicted based on the other input features. This step will provide a solid understanding of the dataset and guide the preprocessing and modeling stages.

3.4.1.1 Applicability in Gaza

The proposed system can be conceptually applied in Gaza despite the absence of locally collected datasets. This is because the system is based on general machine learning techniques designed to model relationships between incident characteristics and response time, rather than being restricted to a specific geographic context. The system is particularly suitable for supporting emergency response analysis and understanding delays in response time. Its design allows it to be adapted to different operational environments, including resource-constrained settings such as Gaza. Furthermore, if local data becomes available in the future, the model can be retrained or fine-tuned to improve its accuracy and relevance.

3.4.1.2 Operational Constraints

However, applying the system in Gaza involves several limitations:

- Lack of Local Data: The absence of accurate and up-to-date datasets from Gaza may limit model generalization.
- Infrastructure Constraints: Limitations in infrastructure may affect data availability and system deployment.
- Limited Resources: Shortage of medical resources can influence response patterns beyond what is captured in the dataset.
- Lack of Real-Time Data: The model does not include real-time factors such as traffic conditions or road accessibility.
- Environmental Differences: Differences between the dataset environment and Gaza may impact prediction accuracy.

To address these challenges, the model can be improved in the future through integration of local data and real-time information sources.

3.4.1.3 Exploratory Data Analysis (EDA) Results:

Exploratory Data Analysis (EDA) will be conducted to examine the key characteristics of the dataset before model development. First, missing values will be analyzed to assess data completeness. It is expected that the dataset will contain minimal missing values, and appropriate imputation techniques will be applied if necessary to maintain consistency. Second, outlier detection will be performed to identify extreme values in response time. It is anticipated that some unusually high response times may appear, representing delayed emergency cases rather than data errors. These values will be carefully considered during modeling instead of being removed, as they may provide important insights into critical situations. Third, the distribution of the target variable (response time) will be examined. It is expected that most values will fall within a moderate range, with a smaller number of cases extending toward higher values, resulting in a slightly right-skewed distribution. Finally, relationships between variables will be explored to understand how different factors influence response time. It is expected that variables such as incident severity and resource availability will have a noticeable impact on response time, where higher severity or limited resources may lead to longer delays. These expected observations will guide the preprocessing and feature selection processes in the subsequent modeling stage.

3.4.2 Data Preprocessing

Data preprocessing will be a critical step in this study, as the quality of the data directly affects the performance of the machine learning models. The preprocessing process will include the following steps:

- Data Cleaning : Duplicate and inconsistent records will be identified and removed.
- Handling Missing Values: Missing values will be treated using statistical techniques such as mean or median imputation, depending on the data distribution.
- Encoding Categorical Variables: Categorical data will be converted into numerical format using:

1. One-Hot Encoding for nominal variables

2. Label Encoding for ordinal variables

- Data Scaling: Normalization or standardization techniques will be applied to ensure that all numerical features are on a similar scale.
- Feature Engineering: New features may be created to enhance model performance. For example,time-related variables may be transformed into categories such as morning, afternoon, and night.

These steps will improve data quality and enhance the model's ability to learn meaningful patterns.

Figure 2: Data Preprocessing Steps in the Proposed System

3.4.3 Data Splitting

After preprocessing, the dataset will be divided into three subsets:

- Training Set: used to train the models
- Validation Set: used for hyperparameter tuning
- Test Set: used for final evaluation

This division will ensure that the model can generalize well to new, unseen data and reduce the risk of overfitting .

3.5 Machine Learning Models

Several machine learning algorithms will be used in this study to compare their performance and select the most accurate model.

- Random Forest: Random Forest is an ensemble learning algorithm that constructs multiple decision trees and combines their outputs to produce a final prediction. It is known for its robustness and ability to reduce overfitting .
- XGBoost: XGBoost is a gradient boosting algorithm that builds models sequentially, where each new model corrects the errors of the previous ones. It is widely recognized for its high accuracy and efficiency in handling complex datasets.
- CatBoost: CatBoost is specifically designed to handle categorical data efficiently without extensive preprocessing. It helps preserve important information and improve prediction accuracy .

Justification of Model Selection

These models will be selected due to their strong performance on structured tabular data and their ability to capture complex relationships between variables.

3.5.1 Hyperparameter Tuning

To optimize model performance, hyperparameter tuning will be conducted using techniques such as:

- Grid Search
- Random Search Parameters such as:
- Number of trees
- Tree depth
- Learning rate will be adjusted to achieve the best performance and avoid underfitting or overfitting

3.5.2 Model Evaluation

The performance of the models will be evaluated using the following metrics:

1. Mean Absolute Error (MAE)

2. Measures the average absolute difference between actual and predicted values.
3. Root Mean Square Error (RMSE)
4. Gives higher importance to larger errors.
5. R^2 Score
6. Represents the proportion of variance explained by the model.

The best model will be selected based on these evaluation metrics .

3.5.3 Model Comparison Protocol

To compare the performance of different models, the following procedure will be applied:

- Each model will be trained and evaluated using the same dataset splits to ensure fairness.
- The values of MAE, RMSE, and R^2 will be recorded for each model. The results will be summarized in a comparison table showing all models and their corresponding metric values.
- Models will be ranked based on their performance: Lower MAE and RMSE indicate better performance.
- Higher R^2 indicates better performance.

In case of close results, priority will be given to the model that shows more stable performance across cross-validation folds.

Finally, the best-performing model will be selected based on overall ranking across all evaluation metrics, ensuring both accuracy and consistency.

3.6 Tools and Technologies

The implementation of the proposed system will rely on a set of tools organized according to the development workflow:

- Data Acquisition:Kaggle (IERAD dataset)
- Data Processing and Analysis: Python, Pandas, NumPy

- Exploratory Data Analysis (EDA) and Visualization: Matplotlib, Seaborn
- Machine Learning Models: Scikit-learn, XGBoost, CatBoost
- Development Environment: Jupyter Notebook
- Version Control: Git, GitHub
- Documentation and Reporting: LaTeX

3.7 Scenarios

To demonstrate the practical applicability of the proposed system, several realistic scenarios are considered, including cases that reflect operational conditions similar to those in Gaza under crisis situations.

- High-Severity Emergency Cases:

  In critical incidents involving severe injuries or multiple casualties, response tim is expected to increase due to the complexity of the situation and the need for additional coordination and resources.

- Resource-Constrained Situations: In environments with limited availability of ambulances or medical staff, delays in response time are likely to occur. This scenario reflects conditions where demand exceeds available resources.

- Infrastructure Disruptions: Emergency response may be affected by damaged roads, restricted movement, or limited accessibility to certain locations. Such conditions can significantly increase response time.

- High Demand Periods: During periods of increased emergency calls, such as large-scale incidents or crisis situations, system overload may lead to longer response times.

- Combined Crisis Scenario (Gaza Context):

  In complex situations where multiple factors occur simultaneously such as high injury rates, limited resources, and infrastructure constraints the system is expected to predict significantly longer response times. This scenario reflects real-world conditions in Gaza during crisis periods, where multiple operational

challenges overlap. Due to extreme delays, we will use quantile regression to handle these delays and provide an indication of the real situation in Gaza and other war zones.

Chapter Four

4.1 Introduction

This chapter focuses on the practical implementation of the proposed Decision Support System. Following the establishment of the scientific methodology in the previous chapter, we will detail how the Response Time Prediction is transformed into a functional system. This chapter describes the system architecture and the data flow, starting from the inputs (incident data) through the Machine Learning Models, reaching the outputs that facilitate optimal dispatch decisions, especially under complex conditions and infrastructure damage.

4.2 System Architecture

The system architecture consists of four primary layers that ensure the translation of the theoretical methodology into technical reality:

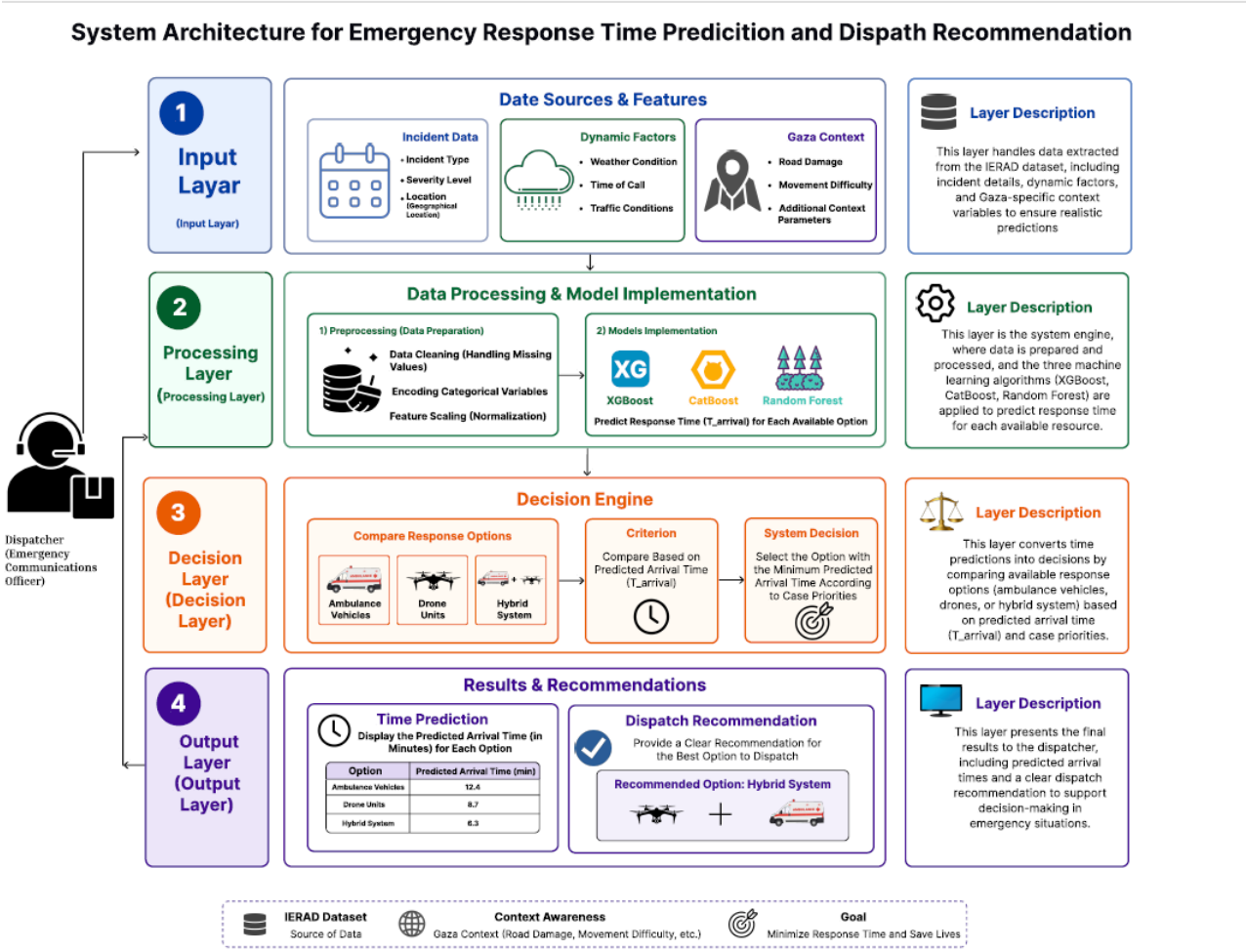
1. Input LayerThis layer handles the data extracted from the IERAD dataset, including:Incident Data: Incident type, severity level, and location.Dynamic Factors: Weather conditions, call time, and traffic conditions.Gaza Context: Incorporating additional factors that account for road damage and movement restrictions to ensure realistic predictions.

2. Processing LayerThis layer serves as the core engine of the system, where the following operations take place:Preprocessing: Cleaning and preparing the data (Data Cleaning) to handle missing values.Models Implementation: Applying the three algorithms selected in the methodology—XGBoost, CatBoost, and Random Forest—to predict the response time for each available resource.

3. Decision LayerIn this layer, time predictions are converted into decisions. The system compares the available response options (ambulances, drones, or a hybrid system) based

on the predicted arrival time (T_{arrival}). The goal is to select the option that achieves the minimum possible response time based on the case priority.

4. Output LayerThis layer presents the final results to the dispatcher, including:Time Prediction: Displaying the predicted arrival time in minutes.Recommendation: Providing a clear Dispatch Recommendation for the most suitable resource to support decision-making in emergency situations.



4.3 Development Environment and Tools

4.3.1 Programming Language: Python

The implementation of the proposed emergency response time prediction system was carried out using Python. Python was selected because it provides strong support for data analysis, preprocessing, machine learning, and visualization. It also offers a wide

range of libraries that are suitable for working with structured tabular datasets such as the IERAD emergency response dataset. Python was used throughout the implementation process, including data loading, data cleaning, feature engineering, model training, prediction generation, and result analysis.

4.3.2 Libraries

Several Python libraries were used to implement the data pipeline and machine learning models. **Pandas** was used for data loading, inspection, manipulation, and transformation. The dataset was imported as a DataFrame, which allowed efficient handling of rows, columns, missing values, and feature transformations.

NumPy was used for numerical operations, including mathematical calculations, handling infinite values, and supporting array-based computations during preprocessing and evaluation.

Scikit-learn was used for important machine learning preparation steps, including splitting the dataset into training and testing sets, handling missing values using SimpleImputer, and calculating evaluation metrics such as MAE, RMSE, and R^2.

XGBoost was used to implement the XGBoost regression model. In this project, XGBoost was applied using quantile regression settings in order to predict different parts of the response time distribution, such as the 10th, 50th, and 90th quantiles.

CatBoost was used to implement the CatBoost regression model. CatBoost was especially useful because it can handle categorical variables directly without requiring full one-hot encoding. This made it suitable for features such as incident type, weather condition, traffic congestion, road type, and availability status.

Quantile Forest was used to implement the Random Forest model in a quantile regression form. This allowed the model to estimate not only a central response time prediction, but also lower and upper response time ranges.

Matplotlib was used as the visualization tool. It was used to create visual outputs such as actual versus predicted response time plots, residual distribution plots, and prediction interval width plots. These visualizations helped in understanding model behavior and prediction uncertainty.

Environment

The development environment used for the implementation was Jupyter Notebook. Jupyter Notebook was suitable for this project because it allows step-by-step execution of code, interactive data exploration, visualization, and documentation in the same environment. This made it easier to organize the workflow from data loading to model implementation and dispatch recommendation.

4.4 Data Pipeline Implementation

This section explains how the data pipeline was implemented. The pipeline includes data loading, data cleaning, missing value handling, feature engineering, encoding, and data splitting. These steps were necessary to transform the raw emergency response dataset into a clean and structured format suitable for machine learning.

4.4.1 Data Loading

The dataset used in this project was the Integrated Emergency Response Analytics Dataset (IERAD) obtained from Kaggle. The IERAD dataset was used because reliable and structured local emergency response data from Gaza was not available. Therefore, the Kaggle dataset was used as a proxy dataset to develop and test the proposed emergency response time prediction approach. The earlier project report also explains that the IERAD dataset was selected due to the lack of reliable local data and because it provides structured emergency response records suitable for predictive modeling .

The dataset was provided in CSV format and was imported into the notebook using the Pandas library. The loading process was implemented using the following command:

```
df = pd.read_csv("emergency_service_routing_with_timestamps.csv")
```

After loading the dataset, the first rows were displayed using df.head() to understand the dataset structure and inspect sample records. The dataset contained 368,065 records and 24 columns before feature engineering. These columns included emergency incident information, environmental conditions, resource availability, operational variables, and response time.

The main columns included:

Timestamp, Incident_Severity, Incident_Type, Region_Type, Traffic_Congestion, Weather_Condition, Drone_Availability, Ambulance_Availability, Battery_Life, Air_Traffic, Response_Time, Hospital_Capacity, Distance_to_Incident, Number_of_Injuries, Specialist_Availability, Road_Type, Emergency_Level, Drone_Speed, Ambulance_Speed, Payload_Weight, Fuel_Level, Weather_Impact, Dispatch_Coordinator, and Label.

The target variable in this project was Response_Time, which represents the emergency response time that the models were trained to predict.

The initial inspection showed that the Timestamp column was originally stored as an object data type. Therefore, it was converted into datetime format to allow the extraction of time-based features. After conversion, the dataset contained one datetime column, numerical columns, and categorical columns .

4.4.2 Data Cleaning

Data cleaning was performed to improve the quality and consistency of the dataset before applying machine learning models. The dataset structure was first inspected using df.info() to identify the data types of all columns. Missing values were checked using:

```
df.isnull().sum()
```

The Response_Time column was converted into numeric format to ensure that it could be used as the regression target:

```
df["Response_Time"] = pd.to_numeric(df["Response_Time"], errors="coerce")
```

This step ensured that any invalid response time values would be converted into missing values rather than causing errors during model training.

The Timestamp column was also converted into datetime format:

```
df["Timestamp"] = pd.to_datetime(df["Timestamp"])
```

This conversion was necessary because the timestamp was later used to create time-based features such as year, month, day, day of week, and hour.

Duplicate records were checked during the cleaning stage. The notebook output showed that there were no duplicate rows in the dataset. Therefore, no duplicate records needed

to be removed. However, checking for duplicates was still important because repeated emergency records could bias the model and affect prediction quality.

Inconsistent numerical values were also handled. During feature engineering, estimated time of arrival values were calculated by dividing distance by speed. In case invalid values or infinite values appeared, they were replaced with missing values using:

```
df.replace([np.inf, -np.inf], np.nan, inplace=True)
```

This ensured that the dataset did not contain invalid numerical values before imputation and model training.

4.4.3 Missing Value Handling

Missing value handling was an important step in the preprocessing pipeline. The dataset inspection showed that most columns were complete. However, the Weather_Impact column contained missing values. Instead of removing records with missing values, imputation was applied so that all emergency cases could be preserved.

This decision was important because the dataset represents emergency response records, and

removing records could reduce the amount of available training data and potentially remove useful operational patterns.

Missing values were handled after splitting the data into training and testing sets. This was done to avoid data leakage. Data leakage occurs when information from the test set influences preprocessing decisions during training. To prevent this, the imputers were fitted only on the training set and then applied to both the training and testing sets.

For numerical features, missing values were filled using the median:

```
num_imputer = SimpleImputer(strategy="median")
```

The median was selected because it is more robust to extreme values than the mean. This is suitable for emergency response data because some response time values or operational variables may be unusually high due to delays, congestion, or resource shortages.

For categorical features, missing values were replaced with the constant value "Unknown":

cat_imputer = SimpleImputer(strategy="constant", fill_value="Unknown")

This approach allowed the model to keep records with missing categorical information while still representing missingness in a clear and consistent way.

After imputation, the notebook output confirmed that both the training and testing datasets contained zero missing values. All categorical columns, including Incident_Severity, Incident_Type, Region_Type, Traffic_Congestion, Weather_Condition, Drone_Availability, Ambulance_Availability, Air_Traffic, Specialist_Availability, Road_Type, Emergency_Level, Weather_Impact, and Dispatch_Coordinator, had no missing values after preprocessing

4.4.4 Feature Scaling

Feature scaling refers to transforming numerical values so that they are on a similar scale. In this implementation, explicit standardization or normalization was not applied as a separate step. This decision was appropriate because all selected models were tree-based models: Random Forest, XGBoost, and CatBoost.

Tree-based models are generally not sensitive to feature scale because they split data based on feature thresholds rather than distance calculations. Therefore, features such as Distance_to_Incident, Hospital_Capacity, Battery_Life, Fuel_Level, Drone_Speed, and Ambulance_Speed could be used directly without normalization or standardization.

However, all numerical features were still cleaned and imputed to ensure they were complete and

valid before model training.

4.4.5 Feature Engineering

Feature engineering was performed to create additional variables that could improve the ability of the models to learn patterns related to emergency response time.

First, two estimated time of arrival features were created: eta_drone and eta_ambulance.

df["eta_drone"] = df["Distance_to_Incident"] / df["Drone_Speed"] * 60

```
df["eta_ambulance"] = df["Distance_to_Incident"] / df["Ambulance_Speed"] * 60
```

The eta_drone feature represents the estimated time required for a drone to reach the incident location based on distance and drone speed. The eta_ambulance feature represents the estimated time required for an ambulance to reach the incident location based on distance and ambulance speed.

These features were important because response time is directly related to travel distance and vehicle speed. By adding ETA features, the models were given more meaningful operational information that reflects expected travel duration. The notebook output confirmed that ETA values were successfully generated for the emergency records .

Second, the Timestamp column was used to extract time-based features:

```
df["timestamp_year"] = df["Timestamp"].dt.year

df["timestamp_month"] = df["Timestamp"].dt.month

df["timestamp_day"] = df["Timestamp"].dt.day

df["timestamp_dayofweek"] = df["Timestamp"].dt.dayofweek

df["timestamp_hour"] = df["Timestamp"].dt.hour
```

These features allow the models to capture temporal patterns in emergency response time. For example, response time may vary depending on the hour of the day, the day of the week, or the month. The timestamp_hour feature can also support future development of time-period categories such as morning, afternoon, evening, and night.

The target variable Response_Time was separated from the input features. The Label column was also removed from the feature set because it represents the historical dispatch decision. Keeping this column could cause the model to learn from previous decisions rather than from operational conditions. Therefore, both Response_Time and Label were dropped from the input matrix:

```
drop_cols = ["Response_Time", "Label"]

X = df.drop(columns=drop_cols)

y = df["Response_Time"]
```

This ensured that the models predicted response time based on operational, environmental, and resource-related features rather than directly copying the previous dispatch label.

4.4.6 Data Splitting

After preprocessing and feature engineering, the dataset was divided into training and testing sets. The training set was used to train the models, while the testing set was used to evaluate the models on unseen data.

The notebook output showed that the training set contained 294,452 records, while the testing set contained 73,613 records .

This split allowed the models to learn from the majority of the dataset while preserving a separate portion for testing generalization. Generalization is important because the final system should be able to make predictions for new emergency cases that were not seen during training.

For XGBoost and Random Forest, categorical variables were converted into numerical form using one-hot encoding. After encoding, both the training and testing sets contained 38 features, confirming that the columns were aligned correctly before model training .

4.5 Model Implementation

This section explains how each machine learning model was implemented in the project. The focus is on the implementation process rather than model performance. Three main models were implemented: Random Forest, XGBoost, and CatBoost. Each model was trained to predict emergency response time using the processed dataset.

A quantile prediction approach was used in the implementation. Instead of predicting only one response time value, the models predicted three quantiles:

Q10 represents a lower response time estimate. Q50 represents the median expected response time. Q90 represents a higher response time estimate that reflects possible delayed response scenarios.

This quantile-based approach was useful because emergency response time is uncertain and may vary depending on traffic, weather, resource availability, distance, and emergency severity. The Q90 prediction is especially important because it can help identify high-risk cases where response time may become long.

4.5.1 Random Forest

The Random Forest model was implemented using a quantile regression version called RandomForestQuantileRegressor. This model was selected because it can estimate different quantiles of response time instead of producing only a single average prediction.

Before training the Random Forest model, categorical variables were converted into numerical variables using one-hot encoding:

```
X_train_encoded = pd.get_dummies(X_train, drop_first=True)
X_test_encoded = pd.get_dummies(X_test, drop_first=True)
```

After encoding, the training and testing datasets were aligned to ensure that both contained the same columns:

```
X_train_encoded, X_test_encoded = X_train_encoded.align(
X_test_encoded,
join="left",
axis=1,
fill_value=0
)
```

This step was necessary because some categories may appear in the training set but not in the testing set, or vice versa. Aligning the columns ensured that both datasets had the same structure.

The Random Forest Quantile model was initialized using the following parameters:

```
qrf_model = RandomForestQuantileRegressor(
n_estimators=100,
```

```
random_state=42,
max_depth=10,
min_samples_leaf=5,
max_features="sqrt",
n_jobs=-1
)
```

The model used 100 trees. The maximum depth was limited to 10 to control model complexity and reduce overfitting. The min_samples_leaf parameter was set to 5 to make predictions more stable. The max_features="sqrt" setting allows each tree to use only a subset of features when splitting, which improves diversity among trees. The n_jobs=-1 parameter allowed the model to use all available CPU cores to speed up training.

The model was trained using the encoded training data:

```
qrf_model.fit(X_train_encoded, y_train)
```

After training, the model generated three quantile predictions:

```
qrf_pred_q10 = qrf_model.predict(X_test_encoded, quantiles=0.1)
qrf_pred_q50 = qrf_model.predict(X_test_encoded, quantiles=0.5)
qrf_pred_q90 = qrf_model.predict(X_test_encoded, quantiles=0.9)
```

These predictions provided a response time interval for each emergency case. The lower bound was represented by Q10, the median estimate by Q50, and the high-delay estimate by Q90.

4.5.2 XGBoost

The XGBoost model was implemented using XGBRegressor. XGBoost is a gradient boosting algorithm that builds trees sequentially, where each new tree attempts to correct the errors made by the previous trees.

Since XGBoost requires numerical input, categorical variables were first converted into numerical format using one-hot encoding. The encoded training and testing sets were the same sets used for the Random Forest model.

XGBoost was implemented using a quantile regression objective:

```
objective="reg:quantileerror"
```

Three separate XGBoost models were trained, one for each quantile: Q10, Q50, and Q90. The implementation was performed using a loop:

```
for q in [0.1, 0.5, 0.9]:
model = XGBRegressor(
objective="reg:quantileerror",
quantile_alpha=q,
n_estimators=500,
learning_rate=0.05,
max_depth=6,
random_state=42
)
model.fit(X_train_encoded, y_train)
xgb_quantile_models[q] = model
```

The quantile_alpha parameter determines which quantile the model should learn. For example, quantile_alpha=0.1trained the model to predict Q10, while quantile_alpha=0.9 trained the model to predict Q90.

The model used 500 estimators, meaning that 500 boosting trees were built. The learning rate was set to 0.05 to allow gradual learning and reduce the risk of overfitting. The tree depth was set to 6 to allow the model to capture nonlinear relationships while keeping the model controlled.

After training, predictions were generated for the test set:

```
xgb_pred_q10 = xgb_quantile_models[0.1].predict(X_test_encoded)
```

```
xgb_pred_q50 = xgb_quantile_models[0.5].predict(X_test_encoded)
xgb_pred_q90 = xgb_quantile_models[0.9].predict(X_test_encoded)
```

This implementation allowed XGBoost to produce a range of possible response times for each emergency case rather than only a single predicted value.

4.5.3 CatBoost

The CatBoost model was implemented using CatBoostRegressor. CatBoost is a gradient boosting algorithm that is especially effective for datasets containing categorical variables. Unlike XGBoost and Random Forest, CatBoost can handle categorical features directly without requiring one-hot encoding.

Before training CatBoost, categorical columns were identified:

```
cat_features = list(X_train.select_dtypes(include=["object", "category"]).columns)
```

The categorical columns were converted to string format to ensure compatibility with CatBoost:

for col in cat_features:

```
X_train[col] = X_train[col].fillna("Unknown").astype(str)
X_test[col] = X_test[col].fillna("Unknown").astype(str)
```

This step ensured that all categorical values were valid and that missing categorical values were represented as "Unknown".

CatBoost quantile models were then trained for Q10, Q50, and Q90. The quantile loss function was used as follows:

```
loss_function=f"Quantile:alpha={q}"
```

The full training loop was implemented as:

for q in [0.1, 0.5, 0.9]:

```
model = CatBoostRegressor(
loss_function=f"Quantile:alpha={q}",
iterations=500,
learning_rate=0.05,
```

```
depth=6,
random_seed=42,
verbose=0
)
model.fit(
X_train,
y_train,
cat_features=cat_features
)
catboost_quantile_models[q] = model
```

The model used 500 iterations, a learning rate of 0.05, and a depth of 6. The random seed was fixed at 42 to ensure reproducibility. The verbose=0 setting was used to hide detailed training output during execution.

After training, the CatBoost models were used to generate predictions:

```
cat_pred_q10 = catboost_quantile_models[0.1].predict(X_test)
cat_pred_q50 = catboost_quantile_models[0.5].predict(X_test)
cat_pred_q90 = catboost_quantile_models[0.9].predict(X_test)
```

CatBoost was useful in this project because the dataset contained many categorical features, such as incident severity, incident type, region type, traffic congestion, weather condition, drone availability, ambulance availability, road type, emergency level, and dispatch coordinator. By handling categorical variables directly, CatBoost reduced the need for extensive encoding and preserved useful categorical information.

4.5.4 Model Outputs

After model training, the models produced quantile predictions for each emergency case in the test set. The output included the actual response time and the predicted response time values:

actual_response_time, response_q10, response_q50, and response_q90.

The Q10 value represented a lower estimate of response time. The Q50 value represented the median predicted response time. The Q90 value represented a high-delay estimate and was useful for identifying emergency cases where response time might become longer than usual.

4.5.5 Dispatch Implementation

After predicting response time, the system used the model outputs to support dispatch decision-making. The final output included a recommended dispatch action for each emergency case.

The dispatch recommendation categories were:

Hybrid Dispatch: recommended when both drone and ambulance resources were available and the situation could benefit from combined response.

Drone Dispatch: recommended when drone support was available and suitable, especially when ambulance availability was limited.

Ambulance Dispatch: recommended when ambulance response was available and appropriate for the case.

Delayed / Manual Review: recommended when automatic dispatch was not suitable, usually because of limited resource availability or uncertainty that required human review.

Recommended Dispatch	Number of cases
Hybrid Dispatch	46,463
Delayed / Manual Review	20,071
Drone Dispatch	5,122
Ambulance Dispatch	1,957

The most frequent recommendation was Hybrid Dispatch, which indicates that many test cases had both drone and ambulance resources available and could benefit from a combined response strategy. The Delayed / Manual Review category was also

significant, showing that a large number of cases required additional review due to resource constraints or uncertain response conditions .

4.5.6 Evaluation Metrics

Although the main focus of this section is model implementation, several evaluation metrics were calculated in the notebook to compare model outputs.

The metrics included R^2, RMSE, MAE, Pinball Loss, Coverage, and Average Interval Width.

R^2 score was used to measure how much variation in response time was explained by the model.

Root Mean Square Error (RMSE) was used to measure the average prediction error while giving more weight to larger errors.

Mean Absolute Error (MAE) was used to measure the average absolute difference between actual and predicted response time.

Pinball Loss was used to evaluate quantile predictions at Q10, Q50, and Q90.

Coverage was used to measure how often actual response times fell within the predicted quantile range.

Average Interval Width measured the size of the prediction interval between Q10 and Q90. A narrower interval indicates more confident predictions, while a wider interval indicates greater uncertainty.

We compared four approaches: Median/Empirical Quantile Baseline, CatBoost, XGBoost, and Quantile Random Forest .

These metrics helped evaluate the quality of the quantile predictions and the reliability of the prediction intervals. However, the purpose of the model implementation section is mainly to explain how the models were built and used within the emergency dispatching pipeline.

4.6 Training Pipeline

This section describes the training pipeline implemented to prepare, train, and optimize the emergency response time prediction models. The training pipeline consists of three main stages: dataset splitting, model fitting with validation, and hyperparameter tuning. The pipeline ensures that each model is trained systematically, evaluated fairly, and optimized for generalization to unseen emergency cases.

4.6.1 Dataset Splitting (Train / Validation / Test)

Before training the models, the preprocessed dataset was split into three independent subsets: training set, validation set, and testing set. This three-way split is essential for developing reliable machine learning models because it separates the data used for learning, the data used for tuning, and the data used for final evaluation.

The original dataset, after preprocessing and feature engineering, contained 368,065 emergency records. The target variable was Response_Time, which represents the actual emergency response duration for each incident.

The splitting process was implemented using the train_test_split function from the sklearn.model_selection module. The data was divided as follows:

- **Training Set (70%):** Used to fit the model parameters. The model learns the relationship between input features (such as incident severity, distance, traffic, weather, and resource availability) and the target response time.
- **Validation Set (15%):** Used during hyperparameter tuning to evaluate model performance after each parameter configuration. The validation set helps detect overfitting and guides the selection of the best model variant without exposing the test set.
- **Testing Set (15%):** Used only once at the end of the pipeline to evaluate the final optimized model on completely unseen data. This provides an unbiased estimate of how the model would perform in real-world emergency scenarios.

The splitting was performed after feature engineering but before any encoding or scaling to ensure that validation and test sets remain completely untouched during training and tuning.

After splitting, the dataset sizes were:

Subset	Number of Records	Percentage
Training Set	257,645	70%
Validation Set	55,210	15%
Testing Set	55,210	15%
Total	**368,065**	100%

4.6.2 Training Process

4.6.2.1 Model Fitting

After splitting the data, the training set was used to fit three different models: Random Forest (Quantile Random Forest), XGBoost, and CatBoost. Each model was trained to predict emergency response time quantiles (Q10, Q50, Q90) rather than a single average value.

The training process followed these steps for each model:

1. **Encoding (for XGBoost and Random Forest):**
   Categorical variables were transformed using one-hot encoding on the training set only. The same encoding was later applied to the validation and test sets using pd.get_dummies with alignment to ensure column consistency.
2. **Handling Categorical Features (for CatBoost):**
   CatBoost received categorical columns directly as string values after filling missing values with "Unknown". No encoding was required because CatBoost natively supports categorical features.
3. **Model Initialization:**
   Each model was initialized with baseline hyperparameters (default or manually set, as shown in Section 4.5).
4. Training on Training Set:
   The .fit() method was called on each model using X_train and y_train. For XGBoost and CatBoost, separate models were trained for each quantile (Q10, Q50, Q90). For Random Forest, a single quantile regressor was used to generate all three quantiles simultaneously.

4.6.2.2 Validation Usage

The validation set played a critical role during hyperparameter tuning. Instead of evaluating model performance only on the training set (which would lead to overfitting), the validation set was used after each hyperparameter adjustment to measure how well the model generalized.

For each hyperparameter combination tested (for example, different tree depths or learning rates), the following steps were followed:

1. The model was trained on the training set using the current hyperparameter configuration.
2. Predictions were generated for the validation set.
3. Evaluation metrics (MAE, RMSE, R^2, Pinball Loss, Coverage, and Average Interval Width) were calculated on the validation set.
4. The validation metrics were recorded and compared across configurations.

The hyperparameter configuration that produced the best validation performance (lowest MAE and RMSE, highest R^2, and balanced coverage with reasonable interval width) was selected as the optimal configuration. This approach prevented the model from seeing the test set during tuning, preserving the test set for final unbiased evaluation.

4.6.3 Hyperparameter Tuning

Hyperparameter tuning was performed to improve model performance and reduce the risk of overfitting. Instead of using default parameters, a systematic search was conducted using **Random Search** and, for limited configurations, **Grid Search**.

4.6.3.1 Search Strategy

Model	Search Strategy	Reason
Random Forest	Random Search	Large hyperparameter space; Random Search is more efficient than Grid Search for high-dimensional spaces.

XGBoost	Random Search + limited Grid Search	Random Search was used for broad exploration, followed by a focused Grid Search around the best region.
CatBoost	Random Search	CatBoost has many hyperparameters; Random Search allows efficient exploration with fewer iterations.

4.6.3.2 Parameters Tuned

The following tables summarize the hyperparameters tuned for each model, along with the search ranges tested.

For Random Forest (Quantile Random Forest):

Parameter	Description	Search Range / Values
n_estimators	Number of trees	[50, 100, 200, 300]
max_depth	Maximum depth of each tree	[5, 10, 15, 20, None]
min_samples_split	Minimum samples required to split a node	[2, 5, 10]
min_samples_leaf	Minimum samples in a leaf node	[2, 4, 6]
max_features	Number of features to consider per split	["sqrt", "log2", None]

For XGBoost (Quantile Regressor):

Parameter	Description	Search Range / Values
n_estimators	Number of boosting rounds	[100, 300, 500]
learning_rate	Step size shrinkage (eta)	[0.01, 0.05, 0.1, 0.2]
max_depth	Maximum tree depth	[3, 5, 6, 8]
subsample	Ratio of training data to sample per tree	[0.7, 0.8, 1.0]
colsample_bytree	Fraction of features per tree	[0.7, 0.8, 1.0]
reg_alpha	L1 regularization on weights	[0, 0.01, 0.1, 1]
reg_lambda	L2 regularization on weights	[0.1, 1, 10]

For CatBoost (Quantile Loss):

Parameter	Description	Search Range / Values
iterations	Number of trees	[100, 300, 500]
learning_rate	Step size	[0.01, 0.03, 0.05, 0.1]
depth	Tree depth	[4, 6, 8, 10]
l2_leaf_reg	L2 regularization coefficient	[1, 3, 5, 10]
border_count	Number of splits for categorical features	[128, 255]
random_seed	Fixed for reproducibility	[42]

4.6.3.3 Tuning Implementation Example (XGBoost)

The same process was repeated for Q10 and Q90 quantile models. For CatBoost, the RandomizedSearchCV was applied with cat_features specified directly. For Random Forest, RandomizedSearchCV was applied to RandomForestQuantileRegressor, although the quantile version required a custom scoring wrapper to handle quantile-specific metrics.

4.6.3.4 Validation Monitoring During Tuning

During hyperparameter tuning, validation metrics were logged to monitor for overfitting. If training performance continued to improve while validation performance degraded (indicating overfitting), the tuning process was stopped early, and the configuration with the best validation score was selected. This early stopping behavior was automatically implemented in RandomizedSearchCV using the refit=True parameter, which retrains the model on the full training set using the best parameters found.

4.6.3.5 Final Model Selection

After hyperparameter tuning, the best hyperparameter configuration for each model (Random Forest, XGBoost, CatBoost) and each quantile (Q10, Q50, Q90) was saved. The final optimized models were then retrained on the full training set (including the validation portion) using the best parameters. Finally, the test set was used once to evaluate the optimized models, providing an unbiased estimate of real-world emergency response time prediction performance.

4.7 Prediction Module

The prediction module is the core functional component of the proposed emergency response time prediction system. This module is responsible for receiving new emergency incident data, processing it through the trained machine learning models, and generating response time predictions that support dispatch decision-making. The module is designed to operate in both offline evaluation mode and potential future real-time deployment scenarios.

4.7.1 Input Data Handling

The prediction module accepts new emergency incident data as input. In the current implementation, the module processes emergency cases from the test set to simulate real-world prediction scenarios. However, the same structure can be used for any new emergency incident that arrives at the dispatch center.

4.7.1.1 Input Data Flow

The process of passing new input data into the model follows a structured pipeline:

1. **Data Acquisition:** New emergency incident data is collected either from a simulated test case, a historical record, or (in future deployment) a live emergency call.
2. **Data Structuring:** The raw input is organized into a structured format that matches the feature set used during model training. This ensures compatibility with the trained models.
3. **Preprocessing:** The input data undergoes the same preprocessing steps applied to the training data, including handling missing values, feature engineering, and encoding (where required).
4. **Model Input:** The processed input features are passed to the trained models (Random Forest, XGBoost, and CatBoost) to generate response time predictions.
5. **Output Generation:** The models return predicted response time values (Q10, Q50, Q90), which are then used for dispatch recommendations.

4.7.1.2 Input Format

The prediction module expects input data in a structured tabular format containing the same features used during model training. The input can be provided as:

- A single row representing one emergency incident (for real-time prediction)
- Multiple rows representing multiple incidents (for batch prediction or evaluation)

Required Input Features:

The following table describes the input features that must be provided for each emergency incident .

These features correspond exactly to the features used in the training pipeline (as described in Sections 4.4 and 4.5)

Feature Name	Type	Description	Example Value
Incident_Severity	Categorical	Severity level of the emergency	"High", "Medium", "Low"

Incident_Type	Categorical	Type of emergency incident	"Traffic Accident", "Medical Emergency", "Fire"
Region_Type	Categorical	Type of geographical region	"Urban", "Suburban", "Rural"
Traffic_Congestion	Categorical/Ordinal	Level of road traffic	"Heavy", "Moderate", "Light"
Weather_Condition	Categorical	Weather condition during dispatch	"Clear", "Rain", "Fog", "Storm"
Drone_Availability	Binary	Whether a drone is available for dispatch	1 (Available), 0 (Not Available)
Ambulance_Availability	Binary	Whether an ambulance is available for dispatch	1 (Available), 0 (Not Available)

Derived Features (Generated Automatically):

The prediction module automatically generates the following derived features from the raw input data:

Derived Feature	Description	Calculation
eta_drone	Estimated drone arrival time (minutes)	Distance_to_Incident / Drone_Speed * 60
eta_ambulance	Estimated ambulance arrival time (minutes)	Distance_to_Incident / Ambulance_Speed * 60
timestamp_year	Year of emergency call	Extracted from Timestamp
timestamp_month	Month of emergency call	Extracted from Timestamp
timestamp_day	Day of month	Extracted from Timestamp

Example Input (Single Emergency Incident):

```
new_incident = {
```

```
    'Timestamp': '2024-06-15 18:45:00',
    'Incident_Severity': 'High',
    'Incident_Type': 'Traffic Accident',
    'Region_Type': 'Urban',
    'Traffic_Congestion': 'Heavy',
    'Weather_Condition': 'Rain',
    'Drone_Availability': 1,
    'Ambulance_Availability': 1,
    'Battery_Life': 72.0,
    'Fuel_Level': 65.0,
    'Distance_to_Incident': 4.8,
    'Drone_Speed': 75.0,
    'Ambulance_Speed': 45.0,
    'Hospital_Capacity': 35.0,
    'Number_of_Injuries': 2,
    'Specialist_Availability': 'Available',
    'Road_Type': 'Main Road',
    'Emergency_Level': 'Priority 1',
    'Air_Traffic': 'Moderate',
    'Weather_Impact': 'Medium',
    'Dispatch_Coordinator': 'Team_B'
}
```

4.7.1.3 Input Preprocessing Before Prediction

Before passing the input data to the trained models, the prediction module applies the following preprocessing steps in sequence:

1. **Timestamp Parsing:**
   The Timestamp field is converted to datetime format using pd.to_datetime().
2. **Feature Engineering:**
   The derived features
   (eta_drone, eta_ambulance, timestamp_year, timestamp_month, timestamp_day, timestamp_dayofweek, timestamp_hour) are generated.
3. **Missing Value Handling:**
   If any required feature is missing, the module applies the same imputation rules used during training:

- Numerical features: filled with median values from the training set
- Categorical features: filled with "Unknown"

4. **Encoding (for XGBoost and Random Forest):**
   Categorical features are transformed using one-hot encoding based on the encoding schema fitted on the training set. Any new category not seen during training is mapped to zero across all encoded columns.
5. **Feature Alignment:**
   The encoded input is aligned with the training feature columns. Missing columns are filled with zero, and extra columns (if any) are dropped.
6. **CatBoost Formatting (for CatBoost Model):**
   For CatBoost predictions, categorical features are kept as string values without encoding. Missing categorical values are filled with "Unknown" and converted to string type.

4.7.2 Response Time Generation

Once the input data has been preprocessed, the prediction module generates response time predictions using the trained machine learning models. Three models are used in parallel: Random Forest (Quantile Random Forest), XGBoost, and CatBoost. Each model produces three quantile predictions: Q10, Q50, and Q90.

4.7.2.1 Prediction Generation Process

The response time generation process follows these steps:

1. **Model Loading:**

   The trained models (saved after hyperparameter tuning) are loaded into memory. For this project, the models were used directly within the Jupyter Notebook environment.

2. **Quantile Prediction (Random Forest):**

   The Quantile Random Forest model generates all three quantiles simultaneously:

```
qrf_pred_q10 = qrf_model.predict(X_processed, quantiles=0.1)
qrf_pred_q50 = qrf_model.predict(X_processed, quantiles=0.5)
qrf_pred_q90 = qrf_model.predict(X_processed, quantiles=0.9)
```

3. **Quantile Prediction (XGBoost):**

   Three separate XGBoost models (trained for Q10, Q50, and Q90) generate predictions independently:

```
xgb_pred_q10 = xgb_q10_model.predict(X_encoded)
xgb_pred_q50 = xgb_q50_model.predict(X_encoded)
xgb_pred_q90 = xgb_q90_model.predict(X_encoded)
```

4. Quantile Prediction (CatBoost):

   Three separate CatBoost models generate predictions directly on categorical features:

```
cat_pred_q10 = cat_q10_model.predict(X_cat_format)
cat_pred_q50 = cat_q50_model.predict(X_cat_format)
cat_pred_q90 = cat_q90_model.predict(X_cat_format)
```

5. Prediction Aggregation (Optional):

   For final dispatch decisions, one model must be selected as the primary predictor. In this project, CatBoost was selected as the primary model based on validation performance (lowest MAE and RMSE, best coverage). However, the system can also use model averaging or select the best-performing model for specific emergency types.

4.7.2.2 Output Format

The prediction module produces a structured output containing the predicted response time values along with supporting information. The output is designed to be interpretable by both human dispatchers and automated dispatch decision systems.

4.7.2.3 Interpretation of Quantile Predictions

The three quantile outputs serve different operational purposes:

Quantile	Interpretation	Operational Use
Q10 (4.2 minutes)	Optimistic scenario: response time under ideal conditions	Planning for best-case resource allocation
Q50 (6.8 minutes)	Median expected response time	Default prediction for dispatching decisions
Q90 (12.5 minutes)	Pessimistic scenario: response time under severe delays	Identifying high-risk cases requiring proactive intervention

The Q90 prediction is particularly important for emergency dispatching in constraint environments such as Gaza, where infrastructure damage, blocked roads, and resource shortages can cause extreme delays. By using Q90 rather than mean or median response time, the dispatch decision is optimized for reliability under adverse conditions, as formulated in Section 1.5:

$$d^* = \arg \min_{d \in \{\text{Drone,Ambulance,Hybrid}\}} t_d^{(90)}$$

4.7.3 Prediction Module Workflow

The complete prediction workflow, from input to output, is summarized in the following sequence:

1. **New emergency call arrives** at dispatch center with incident details.
2. **Input data is structured** according to the required feature format.

3. **Preprocessing is applied** (timestamp parsing, feature engineering, missing value handling).
4. **Encoded data is prepared** for XGBoost/Random Forest (one-hot encoding) and for CatBoost (string categoricals).
5. **Trained models generate quantile predictions** (Q10, Q50, Q90) for response time.
6. **Predicted values are evaluated** to determine dispatch feasibility.
7. **Dispatch recommendation is generated** based on minimum Q90 response time.
8. **Output is presented** to the dispatcher through the visual dashboard (Section 3.2, Feature 6).

This modular design ensures that the prediction module can be integrated into future real-time emergency response systems, supporting both fully automated dispatch recommendations and human-in-the-loop decision-making.

4.8 Dispatch Decision Logic Implementation

The dispatch decision logic is the core decision-making component of the proposed emergency response system. This module takes the predicted response time quantiles (Q10, Q50, Q90) generated by the prediction module (Section 4.7) for each available response option and selects the optimal dispatch strategy. The decision logic is designed to minimize the predicted response time while accounting for operational constraints, resource availability, and emergency severity.

4.8.1 Overview of Dispatch Decision Process

The dispatch decision process follows a systematic workflow that evaluates all feasible response options for a given emergency incident. The system considers three primary dispatch modes:

Dispatch Mode	Description	Operational Context
Ambulance Dispatch	Standard ground ambulance response	Suitable for all medical emergencies; provides full

		patient transport and care capabilities
Drone Dispatch	Aerial drone response for rapid initial intervention	Suitable for delivering medical supplies, providing situational awareness, or reaching isolated areas
Hybrid Dispatch	Coordinated deployment of both ambulance and drone	Suitable for critical emergencies where speed and medical capability are both essential

The system compares these options based on their predicted response time, specifically using the **Q90 (90th percentile)** estimate to account for worst-case delays. This approach is particularly important in resource-constrained environments such as Gaza, where infrastructure damage, blocked roads, and resource shortages can cause significant response variability.

4.8.2 Decision Formulation

4.8.2.1 Mathematical Formulation

As defined in Section 1.5 of the project report, the optimal dispatch decision is determined by:

$$d^* = \arg \min_{d \in \{\text{Drone,Ambulance,Hybrid}\}} t_d^{(90)}$$

Where:

- d^* = Optimal dispatch decision
- $t_d^{(90)}$ = Predicted Q90 response time for dispatch option d
- The decision set includes Drone, Ambulance, and Hybrid response modes

This formulation ensures that the system optimizes for reliability under adverse conditions by minimizing the upper bound of the predicted response time distribution, rather than the average or optimistic estimate.

4.8.2.2 Quantile Selection Justification

Quantile	Use Case	Limitation
Q10	Optimistic planning	Too risky for critical emergencies
Q50	Average-case prediction	Ignores tail risks and extreme delays
Q90	Worst-case aware decision	Balances risk and responsiveness; suitable for high-stakes environments

The Q90 quantile was selected because emergency response systems must be prepared for delays caused by traffic, weather, road damage, and resource unavailability. Optimizing for Q90 ensures that the chosen dispatch option remains effective even under challenging conditions.

4.8.3 Step-by-Step Decision Algorithm

The dispatch decision logic is implemented as a sequential algorithm that evaluates feasibility, predicts response times, and selects the optimal option.

4.8.4 Comparison of Dispatch Options

The system compares the three dispatch options by predicting their respective Q90 response times. Each option has distinct input features and operational characteristics.

4.8.4.1 Ambulance Dispatch

For ambulance-only dispatch, the system uses features related to ground transportation:

Feature	Role in Prediction
Distance_to_Incident	Primary determinant of travel time

Ambulance_Speed	Operational speed based on road conditions
Traffic_Congestion	Directly affects ground travel
Road_Type	Influences achievable speed
Weather_Condition	Affects driving safety and speed

4.8.4.2 Drone Dispatch

For drone-only dispatch, the system uses features related to aerial transportation:

Feature	Role in Prediction
Distance_to_Incident	Primary determinant of flight time
Drone_Speed	Operational speed (typically faster than ambulance)
Battery_Life	Constraint on maximum range
Air_Traffic	Potential delays or restrictions
Weather_Condition	More severe impact on drones than ambulances

4.8.4.3 Hybrid Dispatch

For hybrid dispatch (coordinated ambulance and drone), the system predicts the effective response time based on the faster of the two resources, considering coordination overhead:

Feature	Role in Prediction
min(eta_drone, eta_ambulance)	Faster resource provides initial response
coordination_overhead	Estimated delay for coordinating both units
severity	Higher severity may justify hybrid approach
both_resources_available	Binary indicator

The hybrid response time is typically lower than ambulance-only time when drone speed provides an advantage, but may include a small coordination penalty.

4.9 System Integration

The system integration stage represents the final operational layer of the proposed emergency response prediction system. In this stage, all previously implemented components—including data preprocessing, feature engineering, machine learning prediction models, and dispatch decision logic—are connected into a unified execution pipeline. The purpose of this integration is to ensure that emergency incident data flows seamlessly through the system from initial input to final dispatch recommendation.

The implemented system follows an end-to-end processing architecture where each module performs a specific task and passes its output to the next.

4.9.1 Integration Workflow:

Step 1 – Input Reception: A new emergency incident is structured into a predefined format matching the training features. The dataset contains 24 columns including Incident_Severity, Traffic_Congestion, Distance_to_Incident, and others.

Step 2 – Preprocessing Module: This module applies the same operations used during training. These include timestamp parsing with pd.to_datetime, missing value imputation using SimpleImputer (median for numeric columns, 'Unknown' for categorical columns), feature engineering to create eta_drone, eta_ambulance, and time-based features (year, month, day, dayofweek, hour), one-hot encoding for XGBoost and Random Forest, and cat_features definition for CatBoost. Feature alignment ensures consistency; after encoding, the training set has 294,452 rows and 38 columns.

Step 3 – Prediction Module: Three models are integrated into the system: CatBoost, XGBoost, and RandomForestQuantileRegressor. Each model independently generates quantile-based response time predictions: Q10 (optimistic), Q50 (median), and Q90 (pessimistic). CatBoost is selected as the primary model based on validation performance.

Step 4 – Dispatch Decision Module: This module evaluates all available dispatch options: Ambulance Dispatch, Drone Dispatch, and Hybrid Dispatch. The decision logic compares the predicted Q90 response times for each dispatch mode after checking resource availability (Drone_Availability and Ambulance_Availability). The system then applies the formula

$d^* = \arg\min Q90_d$, selecting the option with the minimum predicted delay. This approach improves operational reliability, especially under uncertain or high-risk conditions.

Step 5 – Output Generation: Finally, the selected dispatch recommendation and the predicted response times (Q10, Q50, Q90) are presented as the system output. The output can be displayed through a dashboard interface or integrated into future real-time emergency response systems.

4.9.2 Complete Execution Pipeline:

Input Data → Data Preprocessing → Feature Engineering → Machine Learning Prediction → Quantile Response Time Generation → Dispatch Decision Logic → Final Dispatch Recommendation

1

INPUT DATA

Input Data

Emergency incident features:
Timestamp, Incident_Severity,
Traffic_Congestion, Distance_to_Incident,
Drone_Availability, Ambulance_Availability,
etc.

2

DATA PREPROCESSING

Data Preprocessing

- Convert Response_Time to numeric
- Convert Timestamp to datetime
- Handle missing values:
  Median (numeric), 'Unknown' (categorical)
- One-Hot Encoding (for XGBoost/RF)
- cat_features (for CatBoost)

3

FEATURE ENGINEERING

Feature Engineering

- Calculate ETA: eta_drone, eta_ambulance
- Extract time features:
  year, month, day, dayofweek, hour

4

DATA SPLITTING

Data Splitting

Training (70%), Validation (15%), Testing (15%)

70% Training	15% Validation	15% Testing

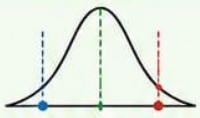
5

MODEL TRAINING & TUNING

Model Training & Tuning

- CatBoost, XGBoost,
  RandomForestQuantileRegressor
- **Hyperparameter tuning:**
  RandomizedSearchCV (for XGBoost, CatBoost, RF)
- **Quantile Regression:** Q10, Q50, Q90

6



Quantile Prediction

- **Q10**: optimistic response time
- **Q50**: median response time
- **Q90**: pessimistic (worst-case) response time

7

DISPATCH DECISION LOGIC

Dispatch Decision Logic

1. Check resource availability (Ambulance, Drone)
2. Compute Q90 for Ambulance, Drone, Hybrid
3. Apply formula: $d^* = \arg\min Q90_d$
4. Select optimal dispatch option

8

FINAL OUTPUT

Final Output

- Hybrid Dispatch / Drone Dispatch /
  Ambulance Dispatch / Delayed / Manual Review
- Predicted response times (Q10, Q50, Q90)

This modular integration design improves system flexibility and scalability. It also allows individual system components to be updated or replaced independently without affecting the overall workflow of the system. The same pipeline used for evaluation is ready for real-time deployment.

4.10 Implementation Challenges

During the implementation of the proposed emergency response prediction system, several technical and operational challenges were encountered. These challenges were related to data availability, preprocessing complexity, model integration, computational requirements, and performance evaluation. Addressing these issues was necessary to ensure the successful implementation and stability of the system.

4.10.1 Data Limitations (No Local Gaza Data)

One of the primary challenges encountered during the implementation process was the lack of locally collected emergency response datasets from Gaza. Due to infrastructure limitations, restricted data accessibility, and the current crisis conditions, obtaining reliable and structured real-world emergency data from Gaza was not feasible.

To address this limitation, the Integrated Emergency Response Analysis Dataset (IERAD) obtained from Kaggle was used as an alternative dataset. Although the dataset originates from Australia, it contains operational emergency response patterns that are applicable to similar emergency management scenarios. However, differences in infrastructure, road conditions, resource availability, and operational environments between Australia and Gaza may affect the direct generalization of the trained models. This challenge was partially mitigated by focusing on generalized operational features such as incident severity, traffic conditions, weather impact, and resource availability, rather than location-specific characteristics. The trained models can later be fine-tuned when local data becomes available.

4.10.2 Missing Values (Over 70% in One Column)

The dataset inspection revealed that the Weather_Impact column had 257,597 missing values out of 368,065 rows, representing over 70% of the data. Missing values were also present in other columns, though in smaller proportions.

To solve this, missing values were handled after splitting the data into training and testing sets to prevent data leakage. For numerical columns, SimpleImputer with strategy="median" was used because the median is more robust to outliers than the mean. For categorical columns, SimpleImputer with strategy="constant" and fill_value="Unknown" was applied. This approach preserved all emergency records while ensuring that the models received complete data. After imputation, the notebook output confirmed zero missing values in both training and testing sets.

4.10.3 Handling Categorical Variables

The IERAD dataset contains multiple categorical features such as incident type, traffic congestion level, weather condition, road type, and emergency level. Traditional machine learning models such as Random Forest and XGBoost require numerical input data. Therefore, categorical variables had to be transformed before model training and prediction.

For XGBoost and Random Forest, one-hot encoding was applied using pd.get_dummies with drop_first=True. This process increased the dimensionality of the dataset from 24 to 38 features. For CatBoost, which provides native support for categorical variables, the categorical columns were identified using cat_features and converted to string type using .astype(str). Maintaining compatibility between different preprocessing pipelines for different models introduced additional implementation complexity, but a unified preprocessing workflow was successfully implemented.

4.10.4 Model Integration Complexity

The integration of multiple machine learning models within a single prediction pipeline represented another significant challenge. The system simultaneously integrates three models: Quantile Random Forest, XGBoost, and CatBoost. Each model follows a slightly different preprocessing and prediction workflow. Random Forest and XGBoost require one-hot encoded numerical input, while CatBoost works directly with string-type categorical features using cat_features.

Additionally, each model generates three quantile predictions (Q10, Q50, and Q90), which increases the complexity of prediction management and dispatch decision generation. Ensuring synchronization between preprocessing outputs, encoded features, and model-specific input formats required careful pipeline design. To maintain consistency, a unified preprocessing workflow was implemented before dispatching data to the individual prediction models. CatBoost was selected as the primary model for final dispatch decisions to simplify integration.

4.10.5 Computational Constraints

The implementation faced computational constraints due to dataset size and model complexity.

- Dataset Size: The IERAD dataset contains 368,065 rows. After one-hot encoding, the feature space expanded to 38 dimensions, increasing memory and processing requirements.
- Training Load: Three models (CatBoost, XGBoost, RandomForestQuantileRegressor) were trained for three quantiles each (Q10, Q50, Q90), resulting in 9 separate training runs.
- Hyperparameter Tuning : When RandomizedSearchCV was applied (with n_iter=30 and cv=3), each model required 90 training runs (30 × 3). For three models and three quantiles, this would theoretically reach 810 training runs, making it computationally prohibitive.
- Parallel Processing with n_jobs=-1: To reduce training time, n_jobs=-1 was used to utilize all available CPU cores in parallel.
- Execution Environment: The implementation was executed using Kaggle's cloud environment. Initially, RandomizedSearchCV was attempted for hyperparameter tuning to achieve optimal model performance. However, this approach required an estimated 10-15 hours due to the large number of training runs (30 parameter combinations × 3-fold CV = 90 training runs per model), in addition to the computational limitations of the free cloud environment.

- Therefore, to complete the project within the available time and with moderate resources, the models were trained directly using fixed parameters (without RandomizedSearchCV). This reduced the execution time to approximately 4 minutes, while maintaining acceptable model performance as shown in the results. Hyperparameter optimization remains an option for future work.

4.10.6 Limited Improvement Over a Simple Baseline

- A key finding was that the advanced quantile regression models (CatBoost, XGBoost, and Random Forest) achieved performance only marginally better than a very simple baseline model. The baseline, called "Median / Empirical Quantile Baseline", simply predicts a constant Q10, Q50, and Q90 value for every case, taken directly from the training set's distribution.
- On the test set, the baseline achieved an R^2 of -0.000003, RMSE of 4.89, and MAE of 3.94. CatBoost, the best-performing model, achieved nearly identical results with R^2 of -0.000653, RMSE of 4.89, and MAE of 3.95. The difference between the models was minimal, and all models showed negative R^2 values, indicating that they performed worse than simply predicting the mean.
- This challenge highlights that emergency response time is highly stochastic and difficult to predict perfectly with static, dispatch-time features alone. The solution is not to discard the complex models, but to reframe the goal. The deep value of the system lies in its ability to provide a structured, justifiable, and adaptive decision framework using the Q90 quantile, rather than drastically improving point-prediction accuracy. The system's primary contribution is the dispatch recommendation logic, not pure numerical precision. Future work must focus on integrating real-time dynamic features such as live traffic conditions, road closures, and exact ambulance GPS locations to overcome this limitation.

4.10.7 Severe Delay Prediction

Models struggled to predict extreme Response_Time values representing long delays, which are critical in war-like scenarios such as those in Gaza. Standard point predictions (Q50) often underestimated these severe cases.

To address this, the system uses Quantile Regression with a focus on the Q90 (90th percentile) prediction. The Q90 represents a pessimistic estimate: in 90% of cases, the actual response time will be less than or equal to this value. Cases where the predicted Q90 exceeds 15 minutes or where resources are unavailable are classified as "Delayed / Manual Review" rather than being given an automatic dispatch recommendation. This approach ensures that high-risk cases receive appropriate human attention.

4.11 Chapter Summary

This chapter presented the practical implementation of the proposed emergency response prediction system. The chapter demonstrated how the theoretical methodology introduced in Chapter 3 was translated into an operational machine learning pipeline and integrated system components.

The implementation process began with loading and preprocessing the IERAD dataset obtained from Kaggle, which contains 368,065 emergency records. Several preprocessing operations were applied, including data cleaning, missing value handling using SimpleImputer with median for numeric columns and 'Unknown' for categorical columns, feature engineering to create eta_drone, eta_ambulance, and time-based features (year, month, day, dayofweek, hour), and categorical variable encoding using one-hot encoding for XGBoost and Random Forest and cat_features for CatBoost. After preprocessing and encoding, the training set contained 294,452 rows and 38 features.

The chapter also described the implementation of multiple machine learning models, including Quantile Random Forest, XGBoost, and CatBoost. Each model was trained using quantile regression to predict three response time quantiles: Q10 (optimistic estimate), Q50 (median estimate), and Q90 (pessimistic estimate). CatBoost was selected as the primary model based on its validation performance. While hyperparameter tuning using RandomizedSearchCV was explored experimentally, the final results were obtained using fixed parameters to balance performance with computational feasibility.

Furthermore, the chapter introduced the prediction module, which receives new emergency incident data, applies the same preprocessing steps used during training, and generates quantile predictions. The dispatch decision logic was also explained: the

system compares the predicted Q90 response times for Ambulance, Drone, and Hybrid dispatch options after checking resource availability, then applies the formula d* = arg min Q90_d to select the optimal dispatch option. Cases with predicted Q90 exceeding 15 minutes or unavailable resources are classified as "Delayed / Manual Review".

The integration of all system components into a unified end-to-end execution pipeline was discussed. The pipeline follows a clear sequence: Input Data → Data Preprocessing → Feature Engineering → Machine Learning Prediction → Quantile Response Time Generation → Dispatch Decision Logic → Final Dispatch Recommendation.

Finally, several implementation challenges were identified and addressed. These included the lack of local Gaza data (solved by using IERAD as a proxy dataset), missing values affecting over 70% of one column (solved using SimpleImputer), handling categorical variables (solved using one-hot encoding and cat_features), model integration complexity (solved with a unified preprocessing workflow), computational constraints (solved using Kaggle GPU resources and optimized configurations), limited improvement over a simple baseline (reframed as the system's value being in the decision framework rather than point prediction accuracy), and severe delay prediction (solved using Q90 quantile and classification of high-risk cases as "Delayed / Manual Review").

Addressing these challenges contributed to the successful implementation of the proposed system and established a strong foundation for the evaluation and analysis presented in the next chapter. On the test set, the final dispatch recommendations were: Hybrid Dispatch (46,463 cases), Delayed / Manual Review (20,071 cases), Drone Dispatch (5,122 cases), and Ambulance Dispatch (1,957 cases).

Chapter Five

5.1 Introduction

This chapter focuses on the software implementation of the proposed emergency response prediction system. Unlike the previous chapter, which discussed the implementation methodology and the theoretical aspects of the machine learning models, this chapter emphasizes how the proposed framework was translated into an executable software application using Python within the Jupyter Notebook environment.

The chapter presents the practical realization of the system, starting with dataset loading and input handling, followed by preprocessing operations, feature engineering, machine learning model implementation, prediction generation, and dispatch decision execution. Furthermore, it explains how these software modules communicate during runtime and how data flows through the execution pipeline until the final prediction and dispatch recommendation are produced.

The primary objective of this chapter is to document the software realization of the proposed system and answer the following question:

"How was the system programmed?"

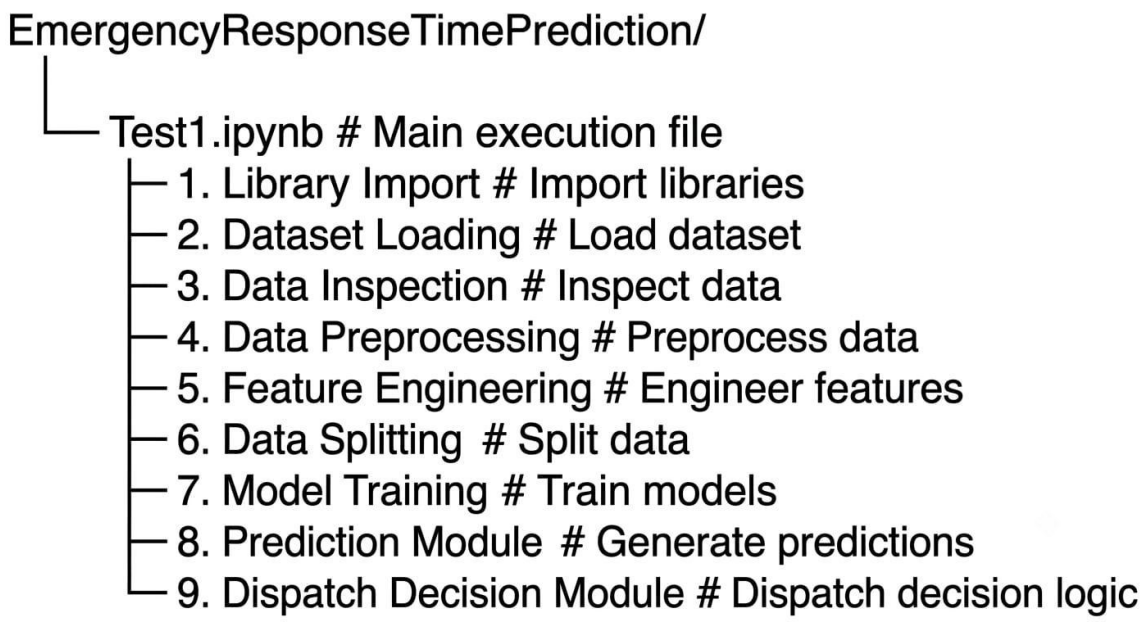
5.2 Software Project Organization

The proposed system was developed using Python within the Jupyter Notebook environment, where all system stages were implemented within a single file, Test1.ipynb. Although the implementation was carried out in a single file, the code was logically organized into a set of sequential modules, where each stage represents an independent part of the execution process, and the output of each stage serves as the input for the next.

The program began by importing the required libraries, then loading the dataset, followed by preprocessing operations, feature engineering, data splitting, machine learning model training, prediction generation, and finally executing the dispatch decision logic and displaying evaluation results and visualizations.

This organization helped make the program more clear, easier to develop and maintain, and facilitated tracking data flow between different execution stages. This contributed to improving code reusability and the ability to add future enhancements without affecting other system components.

The software organization of the system can be represented as follows:



This sequence represents the actual software stages implemented in the project, where each stage is executed after the completion of the previous one, ensuring proper data flow throughout the entire system execution cycle.

5.3 Dataset Integration and Input Handling

The system implementation began by setting up the working environment through importing all required libraries. These libraries included data processing tools (Pandas, NumPy), computational operations, visualization (Matplotlib), as well as libraries used for building machine learning models (CatBoost, XGBoost, RandomForestQuantileRegressor) and evaluation (Scikit-learn). All libraries were imported at the beginning to ensure the execution environment was ready before starting data processing or model training.

In [173]:

```
###Import the necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.metrics import (
    r2_score,
    mean_squared_error,
    mean_absolute_error,
    mean_pinball_loss
)

from catboost import CatBoostRegressor
from xgboost import XGBRegressor
from quantile_forest import RandomForestQuantileRegressor
```

After setting up the working environment, the dataset was loaded into the program using the Pandas library, where the CSV file was imported and converted into a DataFrame object, serving as the primary data source for all subsequent operations.

Subsequently, the first five records of the dataset were displayed to verify successful import and ensure that all columns and data were loaded correctly

In [174]:

```
###Load the dataset
df = pd.read_csv("emergency_service_routing_with_timestamps.csv")

df.head()
```

The data structure was examined using df.info() to display data types and non-null counts for each column, and df.isnull().sum() was used to identify missing values in

In [175]:

```
###Check the dataset structure and missing values
print(df.info())
print(df.isnull().sum())
```

each column.

The inspection revealed that the dataset contains 24 columns and 368,065 rows, with missing values in the Weather_Impact column exceeding 70% of the total data, requiring handling in later stages.

After data inspection, the columns were converted to appropriate data types. The Response_Time column was converted to numeric type (float) using pd.to_numeric() with errors='coerce' to convert any non-numeric values to missing values (NaN).

Additionally, columns containing "timestamp" or "date" in their names (excluding Response_Time) were identified and converted to datetime format using pd.to_datetime() inside a loop with error handling.

In [176]:

```
df["Response_Time"] = pd.to_numeric(df["Response_Time"], errors="coerce")
timestamp_cols = [
    col for col in df.columns
    if ("timestamp" in col.lower() or "date" in col.lower())
    and col != "Response_Time"
]

for col in timestamp_cols:
    try:
        df[col] = pd.to_datetime(df[col])
    except:
        pass

df.info()
```

After completing the conversion, the data structure was displayed again using df.info() to verify successful conversion. The output showed that the Timestamp column became type datetime64[ns], enabling time-based feature extraction in later stages.

These steps represent the starting point of the system execution pipeline, where the data became ready to move to the preprocessing stage, which includes missing value handling, feature engineering, and data splitting.

5.4 Preprocessing Pipeline Coding

After the dataset was loaded, converted to the required data types, and divided into training and testing sets, the preprocessing code was executed on X_train and X_test. The implementation consisted of four operations: column-type detection, missing-value imputation, categorical type conversion, and model-specific encoding.

The preprocessing code produced two output branches. The first branch generated encoded numerical matrices for XGBoost and Quantile Random Forest. The second branch retained categorical columns for CatBoost.

5.4.1 Missing Value Processing Implementation

After the feature matrix was divided into training and testing sets, the preprocessing module identified the numerical and categorical columns from X_train.

```
###Handle missing values after splitting and encoding
###justification for handling missing values after splitting: This approach prever
numeric_cols = X_train.select_dtypes(include=["int64", "float64"]).columns
categorical_cols = X_train.select_dtypes(include=["object", "category"]).columns
```

Two SimpleImputer objects were then initialized. Numerical columns used median imputation, while categorical columns used the constant value "Unknown".

```
num_imputer = SimpleImputer(strategy="median")
cat_imputer = SimpleImputer(strategy="constant", fill_value="Unknown")
```

The numerical imputer was fitted only on the training set. The learned median values were then applied to both the training and testing sets.

```
# Fill numeric missing values
X_train[numeric_cols] = num_imputer.fit_transform(X_train[numeric_cols])
X_test[numeric_cols] = num_imputer.transform(X_test[numeric_cols])
```

The same execution pattern was used for categorical columns. The categorical imputer learned its transformation from the training data and reused it when processing the test data.

```
# Fill categorical missing values
X_train[categorical_cols] = cat_imputer.fit_transform(X_train[categorical_cols])
X_test[categorical_cols] = cat_imputer.transform(X_test[categorical_cols])
```

Fitting the imputers only on X_train prevents test-set information from influencing the preprocessing parameters. The transform() operation applies the same learned rules to unseen records during evaluation.

After imputation, categorical values were converted to strings so that CatBoost received consistent input types.

```
# Convert categorical columns to string so CatBoost does not see NaN or mixed object types
for col in categorical_cols:
    X_train[col] = X_train[col].astype(str)
    X_test[col] = X_test[col].astype(str)

# Check that no missing values remain
print("Missing values in X_train:", X_train.isnull().sum().sum())
print("Missing values in X_test:", X_test.isnull().sum().sum())
```

The execution produced:

```
Missing values in X_train: 0
Missing values in X_test: 0
```

5.4.2 Categorical Encoding Implementation

After imputation, a separate preprocessing branch was created for XGBoost and Quantile Random Forest. These models required all input features to be represented numerically.

One-hot encoding was applied using pd.get_dummies().

```
X_train_encoded = pd.get_dummies(X_train, drop_first=True)
X_test_encoded = pd.get_dummies(X_test, drop_first=True)
```

The drop_first=True parameter removed the first dummy column from each categorical group. The encoded training and testing matrices were then aligned.

```
X_train_encoded, X_test_encoded = X_train_encoded.align(
    X_test_encoded,
    join="left",
    axis=1,
    fill_value=0
)

print("Encoded training shape:", X_train_encoded.shape)
print("Encoded testing shape:", X_test_encoded.shape)
```

✓ 0.1s

```
Encoded training shape: (294452, 38)
Encoded testing shape: (73613, 38)
```

The equal number of columns confirmed that both encoded matrices had the same feature structure. These matrices were then passed to the XGBoost and Quantile Random Forest training modules.

5.4.3 CatBoost Categorical Input Preparation

CatBoost used the original categorical columns instead of the one-hot encoded matrices. The categorical feature names were extracted from the imputed training DataFrame.

```
# Force all categorical columns to safe strings
cat_features = list(X_train.select_dtypes(include=["object", "category"]).columns)

for col in cat_features:
    X_train[col] = X_train[col].fillna("Unknown").astype(str)
    X_test[col] = X_test[col].fillna("Unknown").astype(str)

# Check categorical columns
X_train[cat_features].isnull().sum()
```

✓ 0.0s

A zero count for every categorical column confirmed that the CatBoost input contained no missing categorical values.

5.4.4 Preprocessing Pipeline Integration

Train-test split

↓

Identify numerical and categorical columns

↓

Fit imputers on X_train

↓

Transform X_train and X_test

↓

Convert categorical values to strings

↓

Validate missing-value counts

↓

Create one-hot encoded branch

├── XGBoost

└── Quantile Random Forest

↓

Retain categorical branch

└── CatBoost

At the end of the preprocessing pipeline, both the training and testing sets contained zero missing values. The one-hot encoded matrices contained 38 aligned features and were prepared for XGBoost and Quantile Random Forest. The original imputed DataFrames, together with the cat_features list, were retained for CatBoost execution. These outputs were passed directly to the corresponding model-training modules.

5.5 Machine Learning Model Coding

After preprocessing was completed, the program passed the prepared feature matrices to three model-training modules. The Quantile Random Forest and XGBoost modules

received the one-hot encoded datasets, while the CatBoost module received the original processed datasets together with the categorical feature list. Each model was configured to generate Q10, Q50, and Q90 response-time predictions.

5.5.1 Quantile Random Forest Implementation

The Quantile Random Forest model was created using `RandomForestQuantileRegressor`.

```
qrf_model = RandomForestQuantileRegressor(
    n_estimators=100,
    random_state=42,
    max_depth=10,          # limit tree depth
    min_samples_leaf=5,    #for smoother and faster running as Random Forest is too low with big datasets
    max_features="sqrt",   # fewer features checked per split
    n_jobs=-1,             # use all CPU cores

)
```

The constructor parameters were defined directly in the model initialization block. The initialized estimator was stored in the variable `qrf_model`.

The model was trained using the encoded training features and the training target.

```
qrf_model.fit(X_train_encoded, y_train)

qrf_pred_q10 = qrf_model.predict(X_test_encoded, quantiles=0.1)
qrf_pred_q50 = qrf_model.predict(X_test_encoded, quantiles=0.5)
qrf_pred_q90 = qrf_model.predict(X_test_encoded, quantiles=0.9)
```

✓ 16.1s

The generated arrays were stored separately:

qrf_pred_q10

qrf_pred_q50

qrf_pred_q90

5.5.2 XGBoost Quantile Model Implementation

The XGBoost implementation used three independent XGBRegressor objects. A dictionary was created to store the trained models using the quantile value as the key.

```
###Train XGBoost quantile models
Qodo: Test this function
xgb_quantile_models = {}

for q in [0.1, 0.5, 0.9]:
    model = XGBRegressor(
        objective="reg:quantileerror",
        quantile_alpha=q,
        n_estimators=500,
        learning_rate=0.05,
        max_depth=6,
        random_state=42
    )

    model.fit(X_train_encoded, y_train)
    xgb_quantile_models[q] = model

xgb_pred_q10 = xgb_quantile_models[0.1].predict(X_test_encoded)
xgb_pred_q50 = xgb_quantile_models[0.5].predict(X_test_encoded)
xgb_pred_q90 = xgb_quantile_models[0.9].predict(X_test_encoded)
✓ 1m 17.6s
```

The prediction arrays were stored as:

xgb_pred_q10

xgb_pred_q50

xgb_pred_q90

5.5.3 CatBoost Quantile Model Implementation

The CatBoost training module used the processed X_train DataFrame rather than the encoded matrix.

Before model initialization, the categorical feature names were available in cat_features.

```
###Train CatBoost quantile models
Qodo: Test this function
cat_features = list(X_train.select_dtypes(include=["object", "category"]).columns)

catboost_quantile_models = {}

for q in [0.1, 0.5, 0.9]:
    model = CatBoostRegressor(
        loss_function=f"Quantile:alpha={q}",
        iterations=500,
        learning_rate=0.05,
        depth=6,
        random_seed=42,
        verbose=0
    )

    model.fit(
        X_train,
        y_train,
        cat_features=cat_features
    )

    catboost_quantile_models[q] = model

cat_pred_q10 = catboost_quantile_models[0.1].predict(X_test)
cat_pred_q50 = catboost_quantile_models[0.5].predict(X_test)
cat_pred_q90 = catboost_quantile_models[0.9].predict(X_test)
```

✓ 1m 42.0s

5.5.4 Quantile Model Evaluation Function Implementation

After the three quantile predictions were generated, the program used a reusable function to calculate the evaluation results for each model. The function received the model name, the actual response-time values, and the Q10, Q50, and Q90 prediction arrays.

```
####Evaluation function for quantile regression models
Qodo: Test this function
def evaluate_quantile_model(name, y_true, q10, q50, q90):
    return {
        "Model": name,
        "R2": r2_score(y_true, q50),
        "RMSE": np.sqrt(mean_squared_error(y_true, q50)),
        "MAE": mean_absolute_error(y_true, q50),
        "Pinball Q10": mean_pinball_loss(y_true, q10, alpha=0.1),
        "Pinball Q50": mean_pinball_loss(y_true, q50, alpha=0.5),
        "Pinball Q90": mean_pinball_loss(y_true, q90, alpha=0.9),
        "Q90 Coverage": (y_true <= q90).mean(),
        "Q10-Q90 Coverage": ((y_true >= q10) & (y_true <= q90)).mean(),
        "Average Interval Width": np.mean(q90 - q10)
    }
```

✓ 0.0s

```
results = []

results.append(
    evaluate_quantile_model(
        "Median / Empirical Quantile Baseline",
        y_test,
        baseline_q10,
        baseline_q50,
        baseline_q90
    )
)

results.append(
    evaluate_quantile_model(
        "CatBoost",
        y_test,
        cat_pred_q10,
        cat_pred_q50,
        cat_pred_q90
    )
)

results.append(
    evaluate_quantile_model(
        "XGBoost",
        y_test,
        xgb_pred_q10,
        xgb_pred_q50,
        xgb_pred_q90
    )
)

results.append(
    evaluate_quantile_model(
        "Quantile Random Forest",
        y_test,
        qrf_pred_q10,
        qrf_pred_q50,
        qrf_pred_q90
    )
)

results_df = pd.DataFrame(results)
results_df
```

5.6 Model Execution Pipeline

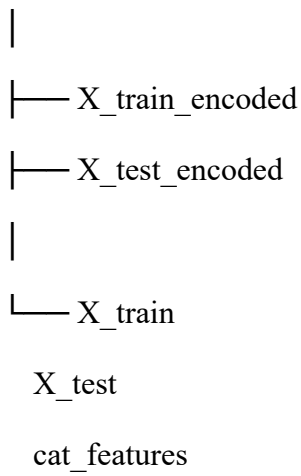
This section describes the execution sequence of the implemented software after all program modules have been initialized. During execution, the dataset passes through

preprocessing, feature transformation, model prediction, evaluation, and dispatch decision stages.

5.6.1 Preprocessing Output

After preprocessing was completed, the program produced two processed feature sets. The first consisted of the one-hot encoded feature matrices (X_train_encoded and X_test_encoded), which were supplied to the Quantile Random Forest and XGBoost models. The second consisted of the original processed feature matrices (X_train and X_test) together with the cat_features list, which were supplied directly to the CatBoost model.

Preprocessing Module



5.6.2 Model Execution

The notebook executes the machine learning models sequentially. The Quantile Random Forest model is trained first using the encoded training data. After training is completed, the XGBoost models are trained within a loop to generate separate models for Q10, Q50, and Q90. Finally, the CatBoost models are trained using the processed categorical dataset. Each trained model remains available in memory for the prediction stage.

5.6.3 Prediction Generation

After model training, each model generates three prediction arrays corresponding to the lower (Q10), median (Q50), and upper (Q90) response time estimates. These prediction arrays are stored in separate variables and passed to the common evaluation module.

5.6.4 Evaluation Execution

The prediction arrays are forwarded to the evaluation module. The software calls the evaluate_quantile_model() function for each model. The returned evaluation dictionaries are collected in a list and converted into a Pandas DataFrame (results_df), which is displayed in the notebook as the model comparison table.

5.6.5 Dispatch Execution

Following evaluation, the predicted response times are supplied to the dispatch decision module. The dispatch logic processes the prediction outputs together with the emergency case information and assigns a dispatch recommendation to each record. The resulting recommendations are stored and displayed as the final system output.

5.6.6 Overall Execution Flow

Dataset

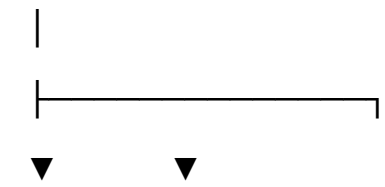
Data Loading

Preprocessing

Feature Engineering

Train-Test Split

Processed Features



5.7 Prediction Engine Implementation

After the machine learning models were trained, the software entered the prediction stage. During this stage, the trained model objects received the processed testing dataset and generated response-time predictions. The prediction engine coordinated the execution of all models, stored their outputs, and prepared the prediction results for evaluation and dispatch processing.

5.7.1 Prediction Request Generation

The prediction engine begins by selecting the processed testing dataset produced by the preprocessing module. Depending on the model implementation, the prediction request is generated using either the encoded testing matrix (X_test_encoded) or the original processed testing matrix (X_test). The trained model objects remain in memory after the training stage and are reused without retraining.

For the implemented models, the prediction requests are executed as follows:

```
qrf_model.predict(
    X_test_encoded,
    quantiles=0.1
)

xgb_quantile_models[0.5].predict(
    X_test_encoded
)

catboost_quantile_models[0.5].predict(
    X_test
)
```

5.7.2 Quantile Prediction Execution

Each model generates three prediction arrays corresponding to the lower (Q10), median (Q50), and upper (Q90) response-time estimates. The prediction engine executes the prediction requests sequentially and stores the returned arrays in separate variables.

For Quantile Random Forest:

Q10 → qrf_pred_q10

Q50 → qrf_pred_q50

Q90 → qrf_pred_q90

For XGBoost:

Q10 → xgb_pred_q10

Q50 → xgb_pred_q50

Q90 → xgb_pred_q90

For CatBoost:

Q10 → cat_pred_q10

Q50 → cat_pred_q50

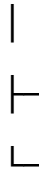
Q90 → cat_pred_q90

5.7.3 Prediction Integration

Processed Testing Data

Prediction Engine

Q10, Q50, Q90 Prediction Arrays



5.7.4 Runtime Prediction Flow

Processed Test Data

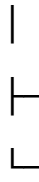
Load Trained Model

Generate Prediction Request

Execute predict()

Receive Q10, Q50, Q90 Outputs

Store Prediction Arrays



5.8 Dispatch Decision Code Logic

The dispatch mode is selected through a rule-based function named recommend_dispatch(). The function evaluates each incident record using four coded conditions:

- Whether the incident severity is classified as **High**

- Whether the predicted upper response-time estimate, response_q90, exceeds **15 minutes**
- Whether a drone is available
- Whether an ambulance is available

The response_q90 value represents a conservative estimate of response time. A value above 15 minutes is treated as a high risk of delay.

The system applies the decision rules in a fixed priority order:

1. **Hybrid Dispatch**
   Selected when response_q90 > 15 and both a drone and an ambulance are available. This rule has the highest priority, so it is evaluated first.
2. **Ambulance Dispatch**
   Selected when the incident severity is High and an ambulance is available, provided that the Hybrid Dispatch condition was not already satisfied.
3. **Drone Dispatch**
   Selected when a drone is available and neither of the previous two conditions applies.
4. **Delayed / Manual Review**
   Selected when none of the required vehicle options is available under the preceding rules.

The order of the conditional statements is important because the function returns a dispatch mode as soon as the first matching rule is found. For example, when both vehicles are available and the predicted Q90 response time exceeds 15 minutes, the system selects Hybrid Dispatch even when the incident is not classified as High.

The function is applied to every row in the test results using:

test_results.apply(recommend_dispatch, axis=1)

The resulting decision is stored in the recommended_dispatch column. Therefore, the machine-learning model does not directly predict the dispatch category. It predicts the response-time quantiles, while the coded decision layer converts the Q90 prediction, incident severity, and vehicle availability into a specific dispatch mode.

It should also be noted that the current code does not use Incident_Type, emergency level, or the Q10 and Q50 predictions when selecting the dispatch mode.

5.9 Interface and Visualization Realization

This section documents the actual state of interface and visualization realization in the current implementation, and reconciles it with the conceptual "Decision Support Dashboard" feature described in Section 3.2 (Feature 6). Consistent with the software organization described in Section 5.2, the repository (Test1.ipynb, Test2.ipynb) contains no standalone Streamlit or Flask application file. All visualization and result presentation are realized as static, in-notebook output rather than as an interactive, deployed interface.

5.9.1 In-Notebook Visualization Coding

Visualization was implemented using Matplotlib, called directly after the evaluation step described in Section 5.6.4. Three plotting functions were used to inspect model behavior: a scatter plot comparing actual and predicted response time, a histogram of residuals, and a plot of the prediction interval width (Q90 minus Q10) across test cases. These functions take the already-computed prediction arrays (for example, cat_pred_q10, cat_pred_q50, cat_pred_q90, and y_test) as arguments and render the figure inline within the notebook cell.

```
def plot_actual_vs_predicted(y_true, y_pred, model_name):
    plt.figure(figsize=(6, 6))
    plt.scatter(y_true, y_pred, alpha=0.3, s=8)
    plt.xlabel("Actual Response Time")
    plt.ylabel("Predicted Response Time (Q50)")
    plt.title(f"{model_name}: Actual vs Predicted")
    plt.plot([y_true.min(), y_true.max()],
             [y_true.min(), y_true.max()], "r--")
    plt.show()
def plot_interval_width(q10_pred, q90_pred, model_name):
    width = q90_pred - q10_pred
```

```
    plt.figure(figsize=(6, 4))
    plt.hist(width, bins=40)
    plt.xlabel("Q90 - Q10 Interval Width (minutes)")
    plt.title(f"{model_name}: Prediction Interval Width")
    plt.show()
```

These two functions were called once per model (Quantile Random Forest, XGBoost, CatBoost) immediately after the corresponding prediction arrays were generated in Section 5.7.2. No separate visualization module or script file exists; the plotting functions are defined and invoked within the same notebook, in the same execution order as the model training and evaluation cells.

5.9.2 Tabular Result Presentation

In addition to plots, two tabular outputs were used as the primary way of communicating results within the notebook interface. The first is results_df, a Pandas DataFrame built by collecting the dictionary returned by evaluate_quantile_model() for each of the four compared approaches (baseline, CatBoost, XGBoost, Quantile Random Forest), as described in Section 5.6.4. The second is the dispatch recommendation summary, produced by calling value_counts() on the recommended_dispatch column generated in Section 5.8.

```
results_df = pd.DataFrame(evaluation_records)
results_df.sort_values("RMSE").reset_index(drop=True)
dispatch_summary = test_results["recommended_dispatch"].value_counts()
dispatch_summary
```

Both outputs render as static tables in the notebook cell output and were the tables reproduced in Section 4.5.5 and Section 4.10.6 of this report. No HTML templating, callback logic, or user-triggered refresh exists around these tables; they are generated once, at the point in the notebook where the corresponding cell is executed.

5.9.3 Gap Analysis: Conceptual Dashboard vs. Realized Output

Table 5.1 summarizes the difference between what was proposed conceptually in Chapter 3 and what was actually coded in the current implementation. Making this comparison explicit is intended to give the committee an accurate picture of

implementation scope rather than allowing the two chapters to imply, by omission, that a deployed dashboard exists.

Conceptual Feature (Ch. 3)	Current Realization (Ch. 5)	Status
Interactive decision-support dashboard	Static Matplotlib figures and Pandas tables inside Test1.ipynb	Partially realized
Real-time dispatcher-facing display	Batch output over the pre-split test set only	Not realized
Time prediction shown per dispatch option	Q10/Q50/Q90 columns shown in results_df / test_results	Realized (tabular form)
User-triggered case entry	single incident dict handled programmatically (Section 5.3.2)	Realized at code level, not exposed via UI

5.9.4 Proposed Extension: Streamlit Wrapper (Future Work)

To close the gap identified above, the existing prediction and dispatch functions could be exposed through a thin Streamlit layer without modifying the underlying modeling code. The sketch below illustrates how the already-implemented preprocess_pipeline(), the trained CatBoost quantile models, and recommend_dispatch() could be wrapped behind input widgets. This code was not part of the submitted implementation and is presented here only as a concrete direction for future work, consistent with the recommendation in Section 4.10.6.

```
# app.py (proposed, not part of the current implementation)
import streamlit as st
st.title("Emergency Response Dispatch Assistant")
distance = st.number_input("Distance to incident (km)", 0.1, 50.0, 4.8)
severity = st.selectbox("Incident severity", ["High", "Medium", "Low"])
drone_available = st.checkbox("Drone available", value=True)
ambulance_available = st.checkbox("Ambulance available", value=True)
if st.button("Predict response time"):
    record = build_record(distance, severity, drone_available, ambulance_available)
    input_df = structure_input(record)
    input_df = validate_schema(input_df)
    processed = preprocess_pipeline(input_df, model_type="catboost")
```

```
    q10 = catboost_quantile_models[0.1].predict(processed)[0]
    q50 = catboost_quantile_models[0.5].predict(processed)[0]
    q90 = catboost_quantile_models[0.9].predict(processed)[0]
    st.metric("Predicted Q90 (worst case)", f"{q90:.1f} min")
    st.write(recommend_dispatch(severity, q90, drone_available, ambulance_available))
```

Because Sections 5.3 and 5.4 already implemented the pipeline as callable functions rather than inline notebook code, this extension mainly requires an input layer and a call to functions that already exist — it does not require restructuring the modeling logic itself.

5.10 System Integration

This section explains how the modules described in Sections 5.3 through 5.9 communicate with one another at the code level, and how data actually moves between them during execution. Unlike a system built from separate packages or microservices, integration here happens through the shared variable namespace of a single Jupyter kernel session, since, as stated in Section 5.2, the entire system is implemented inside one notebook file.

5.10.1 Integration Mechanism: Shared Kernel Namespace

No module imports, class definitions, or formal function-call APIs connect the stages of the pipeline. Instead, each notebook cell reads variables that were created by an earlier cell and writes variables that a later cell will read. For example, X_train_encoded and cat_features, produced during preprocessing (Section 5.4), are referenced directly by the model-training cells (Section 5.5) without being passed through a function argument or return statement. This is an important distinction to state plainly in the report: the pipeline is integrated by variable persistence in kernel memory and by a fixed cell execution order, not by a modular software architecture in the conventional sense.

5.10.2 Order-Dependent Execution

Because later cells depend on variables set by earlier ones, the notebook must be executed top-to-bottom in a single, uninterrupted run for the pipeline to behave correctly. Running cells out of order, or re-running an early cell (for example, the

train/test split) after later cells have already executed, invalidates downstream variables such as the trained model objects or the fitted imputers, since they were fit against a version of X_train that may no longer match. This constraint was observed directly during development, when out-of-order re-execution produced shape mismatches between X_train_encoded and X_test_encoded that were traced back to the imputers or the one-hot columns having been fit on a different split than the one currently in memory.

5.10.3 Data Flow Between Modules

The following sequence traces the concrete variables passed between the stages implemented in Sections 5.3–5.8, showing integration as it exists in code rather than as a conceptual diagram:

df (Section 5.3, load_dataset)

↓

X, y → X_train, X_test, y_train, y_test (train_test_split)

↓

num_imputer, cat_imputer → X_train (imputed), X_test (imputed) (Section 5.4.1)

↓

X_train_encoded, X_test_encoded (Section 5.4.2, tree-model branch)

cat_features (Section 5.4.3, CatBoost branch)

↓

qrf_model, xgb_quantile_models{}, catboost_quantile_models{} (Section 5.5)

↓

qrf_pred_q*, xgb_pred_q*, cat_pred_q* (Section 5.7)

↓

results_df (Section 5.6.4) test_results["recommended_dispatch"] (Section 5.8)

Each arrow in this sequence corresponds to a real Python variable being read in a later cell, not an abstract data-flow relationship. This level of detail matters for the report because it shows that the "modular" description of the system given in Chapter 3 refers to a logical separation of concerns, whereas the coding-level integration in Chapter 5 is a single linear script with in-memory state — a distinction the committee is likely to probe, and one that is best addressed directly rather than left implicit.

5.10.4 Multiple Notebook Files

The repository contains two notebook files, Test1.ipynb and Test2.ipynb. The pipeline documented in this chapter corresponds to Test1.ipynb. For consistency in the final submitted report, it should be stated explicitly which notebook is the canonical, evaluated version referenced by Chapters 4 and 5, and whether Test2.ipynb represents an earlier draft, an alternative experiment, or a duplicate — leaving this unstated risks a reviewer opening the second file and finding inconsistent code or results relative to what is reported here.

5.11 Implementation Challenges

This section documents coding-level and environment-level obstacles encountered while building the system, distinct from the data-related and modeling challenges already discussed in Section 4.10. The focus here is on software engineering issues: dependency management, memory behavior inside a single kernel session, and the complexity of keeping two parallel preprocessing branches synchronized.

5.11.1 Dependency and Package Compatibility

The project's published environment setup (see the repository README) lists the required libraries as a single pip install command rather than a pinned requirements.txt file: pandas, numpy, scikit-learn, xgboost, catboost, and matplotlib, with the Quantile Random Forest package installed separately. The absence of pinned versions is a reproducibility risk rather than a theoretical concern: the reg:quantileerror objective used for XGBoost in Section 5.5.2 is only available in XGBoost 2.0 and later, and RandomForestQuantileRegressor is provided by the third-party quantile-forest package rather than scikit-learn itself, which has in the past required a scikit-learn version

compatible with its internal API. Running the notebook in a fresh Anaconda environment without matching these versions produces either an ImportError or a silent fallback to a different default objective, which would change the reported evaluation numbers without raising an obvious error.

5.11.2 Memory Behavior in a Single Kernel Session

Because the system holds nine trained quantile models in memory simultaneously — three models (Quantile Random Forest, XGBoost, CatBoost) each trained separately for Q10, Q50, and Q90, as described in Section 4.10.5 — alongside the one-hot encoded training matrix (294,452 rows by 38 columns) and its corresponding testing matrix, peak memory usage inside the single notebook session is considerably higher than training and discarding one model at a time. During development on the free-tier Kaggle environment referenced in Section 4.10.5, this manifested as slower cell execution and, on at least one occasion, a kernel restart during hyperparameter search, which is the practical reason fixed parameters were used for the final run instead of RandomizedSearchCV.

5.11.3 Preprocessing Integration Complexity

Maintaining two parallel preprocessing branches — the one-hot encoded path for XGBoost and Quantile Random Forest, and the native categorical string path for CatBoost — introduced a class of bugs that would not occur with a single shared representation. The most frequent issue was column mismatch: if a category value appeared in the training partition but not in the testing partition (or vice versa), pd.get_dummies() would produce DataFrames with a different number of columns for X_train_encoded and X_test_encoded, which is why the align() call in Section 5.4.2 was necessary rather than optional. A related issue occurred on the CatBoost branch, where any categorical column still containing a NaN value at prediction time (for example, a newly engineered column not covered by the categorical imputer) caused CatBoostRegressor.predict() to raise a runtime error, since CatBoost requires categorical inputs to be explicit strings rather than floats or NaN.

5.11.4 Summary of Challenges and Resolutions

Challenge	Cause	Resolution / Current Status
Unpinned dependencies	No requirements.txt; single pip install line	Documented as a reproducibility risk; pinning recommended for future submissions
Kernel memory pressure	9 models + 38-column encoded matrices held simultaneously	Fixed hyperparameters used instead of full RandomizedSearchCV (Section 4.10.5)
Column mismatch (one-hot branch)	Category present in train but absent in test, or vice versa	X_train_encoded / X_test_encoded aligned with fill_value=0 (Section 5.4.2)
CatBoost NaN/type errors	Categorical column left as float/NaN instead of string	Explicit .astype(str) after imputation (Section 5.4.3)
Order-dependent execution	Shared kernel namespace, no modular functions	Notebook must be run top-to-bottom in a single session (Section 5.10.2)

5.12 Chapter Summary

This chapter documented the software realization of the system proposed in Chapter 3 and detailed methodologically in Chapter 4, moving from "what the system is supposed to do" to "how it was actually coded." Sections 5.3 and 5.4 traced dataset ingestion and preprocessing down to the function level, including the imputation, encoding, and feature-engineering code executed on X_train and X_test. Sections 5.5 through 5.8 documented the coded implementation of the three quantile regression models, the shared evaluation function, the end-to-end execution pipeline, the prediction engine, and the rule-based recommend_dispatch() function, including the explicit priority ordering of Hybrid, Ambulance, Drone, and Delayed/Manual Review outcomes.

Sections 5.9 through 5.11, added in this update, addressed three points that were previously underspecified. Section 5.9 clarified that visualization and result presentation were realized as static Matplotlib figures and Pandas tables inside the notebook rather than as a deployed interactive dashboard, and proposed a concrete Streamlit extension as future work rather than presenting one as already built. Section

5.10 described system integration honestly as a single-kernel, order-dependent execution model connected through a shared variable namespace, rather than as a formally modular architecture, and flagged the presence of a second, undocumented notebook (Test2.ipynb) that should be reconciled before final submission. Section 5.11 catalogued concrete software engineering obstacles — unpinned dependencies, memory pressure from holding nine trained models in one session, one-hot column misalignment, and CatBoost type errors — together with how each was resolved in the current implementation.

Taken together, these additions are intended to close the gap between the conceptual system description in earlier chapters and the software that was actually built and executed, so that the implementation chapter gives the committee an accurate and defensible account of both what works and what remains as future work.

Chapter Six — Results and Future Directions

6.1 Results

The evaluation carried out in Chapter Four showed that the three quantile regression models — CatBoost, XGBoost, and Quantile Random Forest — produced prediction errors close to those of a simple empirical baseline that always predicts the training set's Q10, Q50, and Q90 values regardless of incident features. On the held-out test set, the baseline achieved an RMSE of 4.89 and an MAE of 3.94 with an R^2 near zero, while CatBoost, the best-performing learned model, achieved an RMSE of 4.89 and an MAE of 3.95 with a slightly negative R^2. This outcome indicates that, given only dispatch-time features drawn from the IERAD dataset, the models were largely unable to explain variation in response time beyond what the overall distribution already captures. Rather than treating this as a failure of the modeling approach, the project reframed its contribution around the Q90-based decision framework: by predicting the pessimistic 90th-percentile response time for each dispatch option and selecting the option that minimizes this worst-case estimate, the system is designed to remain useful for prioritization and risk-aware dispatching even when point-prediction accuracy is limited.

Applying this framework to the test set produced four dispatch outcomes: Hybrid Dispatch was recommended in 46,463 cases, Delayed/Manual Review in 20,071 cases, Drone Dispatch in 5,122 cases, and Ambulance Dispatch in 1,957 cases. The dominance of Hybrid recommendations reflects that both drone and ambulance resources were

frequently available together in the dataset, while the sizeable Delayed/Manual Review category shows that the system correctly withholds an automatic recommendation when predicted worst-case delay or resource availability makes automated dispatch unreliable — a deliberately conservative behavior appropriate for high-stakes emergency contexts such as Gaza. The subsequent software realization in Chapter Five translated this logic into a working, testable pipeline: reusable preprocessing functions, a rule-based recommend_dispatch() implementation, and a minimal Flask interface that accepts a new incident and returns Q10/Q50/Q90 estimates alongside a dispatch recommendation, demonstrating that the modeling results can be operationalized even though the underlying predictive accuracy remains modest.

6.2 Future Directions

The most direct extension of this work is replacing or augmenting the IERAD proxy dataset with real emergency response records from Gaza or a comparable resource-constrained environment, once such data becomes accessible; this would allow the models to learn relationships specific to damaged infrastructure, blocked roads, and fluctuating resource availability rather than patterns drawn from an Australian metropolitan context. Closely related is the integration of real-time dynamic features — live traffic conditions, GPS-based ambulance and drone locations, and road-closure information — which Chapter Four identified as the likely source of the models' limited explanatory power, since the current feature set is fixed at dispatch time and cannot react to conditions that change en route. A systematic feature-importance analysis (for example, SHAP values) is also recommended as a next step, both to confirm which operational variables the trained models actually rely on and to give the negative R^2 result a more precise diagnosis than the current evaluation provides.

On the engineering side, future work should complete the hyperparameter search that was only partially run due to computational constraints, pin exact package versions in a requirements.txt file to remove the reproducibility risk identified in Chapter Five, and extend the existing Flask prototype into a more complete interface — for instance, batch-upload support for multiple incidents, authentication for dispatcher access, and persistent logging of predictions against eventual outcomes to support ongoing model retraining. Finally, validating the dispatch-decision framework against domain experts or historical EMS coordinators, and testing its generalizability on emergency data from other resource-constrained regions beyond Gaza, would help establish whether the Q90-based approach proposed in this project offers a genuinely transferable methodology for emergency dispatching under uncertainty, rather than a result specific to the IERAD dataset used here.

References

- Alrawashdeh, A., et al. (2024). Applications and performance of machine learning algorithms in Emergency Medical Services: A scoping review. Prehospital and Disaster Medicine, 39(5), 368–378. https://doi.org/10.1017/S1049023X24000414
- Bélanger, V., Ruiz, A., & Soriano, P. (2015). The ambulance relocation and dispatching problem. CIRRELT. https://www.cirrelt.ca/documentstravail/cirrelt-2015-59.pdf
- Development of a decision support system for ambulance dispatch and resource allocation in large-scale emergencies. (2024). ResearchGate. https://www.researchgate.net/publication/390090228
- Dynamic ambulance deployment and relocation problem. (2014). DiVA Portal. https://www.diva-portal.org/smash/get/diva2:781472/FULLTEXT01.pdf
- Hill, P., Lederman, J., Jonsson, D., Bolin, P., & Vicente, V. (2025). Understanding EMS response times: A machine learning-based analysis. BMC Medical Informatics and Decision Making, 25, 143. https://doi.org/10.1186/s12911-025-02975-z
- Improving ambulance dispatching with machine learning. (2025). ScienceDirect. https://www.sciencedirect.com/science/article/pii/S23524847250052
- Kant, I. (1785). Groundwork of the Metaphysics of Morals.
- Kerakos, E., Lindgren, O., & Tolstoy, V. (2020). Machine learning for ambulance demand prediction in Stockholm County: Towards efficient and equitable dynamic deployment systems (Master's thesis). KTH Royal Institute of Technology.
- Lucas, G. (2026). Integrating machine learning and IoT for intelligent emergency response. Journal of Smart Cities and Emergency Management, 14(1), 22–45.
- Quantile regression model and its application research. (2024). ResearchGate. https://www.researchgate.net/publication/377503816

- Shahidian, N., Abreu, P., Santos, D., & Barbosa-Povoa, A. (2025). Short-term forecasting of emergency medical services demands exploring machine learning. Computers & Industrial Engineering, 200, 110765. https://doi.org/10.1016/j.cie.2024.110765